Linear discriminant functions

Plan

1. Linear functions and linear discriminant functions
2. Different classifiers from different distributional assumptions
   - Intermezzo Normal Distribution and $\mathbf{x}^T\mathbf{W}\mathbf{x}$.
3. Fisher's linear discriminants
   - Intermezzo, Rayleigh quotient
4. Perceptron
   - Controlling the learning rate
5. Logistic Regression

Linear Functions I

A linear discriminant (LD) is a linear combination of the components of $\mathbf{x} = \begin{pmatrix} x_2 \\ x_2 \\ \dots \\ x_d \end{pmatrix}$

given by a *weight vector*, $\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_d \end{pmatrix}$,

and a bias parameter, w_0, also known as the *threshold weight*.

Linear Functions II

Then LD can be written as:

$$g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0$$

or as:

$$g(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + w_0$$

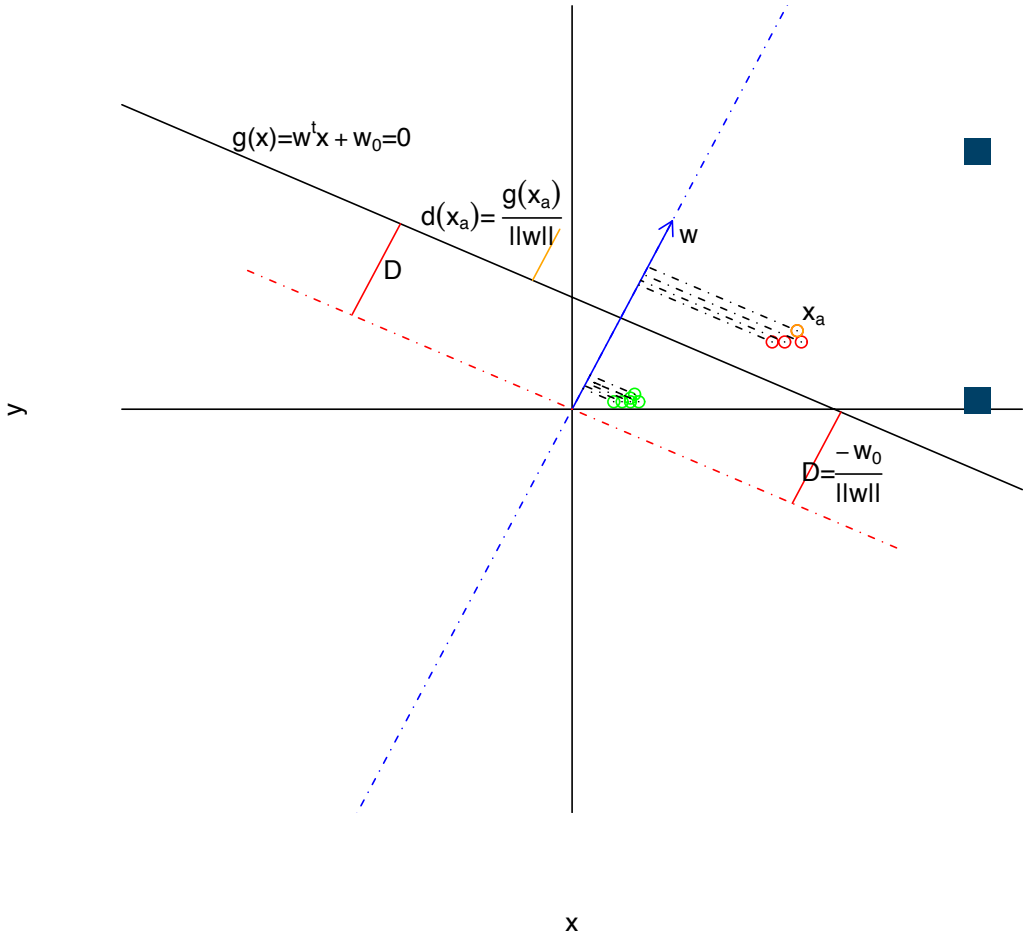
We can also extend the $\mathbf{x}$ vector by a dimension, x_0, fixed to 1, then $\mathbf{x}' := \begin{pmatrix} x_0 \\ \mathbf{x} \end{pmatrix}$ and $\mathbf{w}' := \begin{pmatrix} w_0 \\ \mathbf{w} \end{pmatrix}$ and the LD equation simplifies to:

$$g(\mathbf{x}') = \mathbf{w}'^T\mathbf{x}'$$

Linear Discriminant Functions

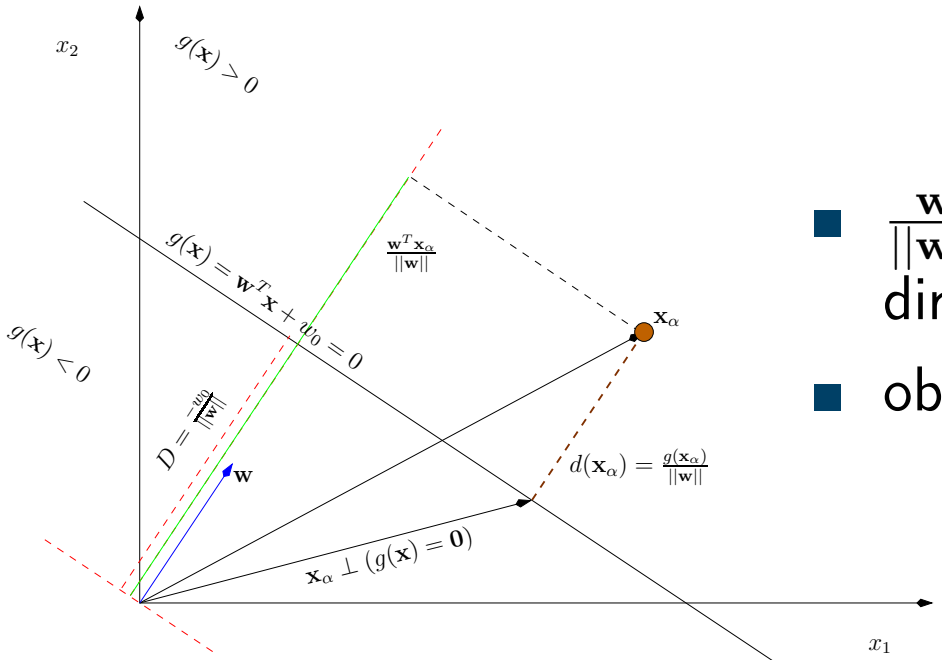
- In the simplest classification scenario we have two classes, ω_1, ω_2, the decision rule of a LD is:
  - If $g(\mathbf{x}) > 0$ then ω_1
  - If $g(\mathbf{x}) < 0$ then ω_2
- $g(\mathbf{x}) = 0$ defines the decision surface that separates the two classes. When $g(\mathbf{x})$ is linear the decision surface, i.e. $g(\mathbf{x})$, is a *hyperplane*.
- $\mathbf{w}$ is the vector that is normal to the hyperplane $g(\mathbf{x}) = 0$

Some notions from geometry I



- $d(\mathbf{x}_a) = \frac{g(\mathbf{x}_\alpha)}{||\mathbf{w}||}$ is the *signed* distance of $\mathbf{x}_\alpha$ from the $g(\mathbf{x}) = 0$ hyperplane.
- $D = -d(\mathbf{x}_a = \mathbf{0}) = -\frac{w_0}{||\mathbf{w}||}$ is the *signed* distance of the $g(\mathbf{x}) = 0$ hyperplane from the origin of the axes.

Distance, $d(\mathbf{x}_\alpha)$, of a point $\mathbf{x}_\alpha$ from the hyperplane $g(\mathbf{x}) = 0$, $d(\mathbf{x}_\alpha) = \frac{g(\mathbf{x}_\alpha)}{||\mathbf{w}||}$



- Lets denote the projection point of $\mathbf{x}_\alpha$ on the decision surface $g(\mathbf{x}) = 0$ by $\mathbf{x}_{\alpha_\perp}$, i.e.

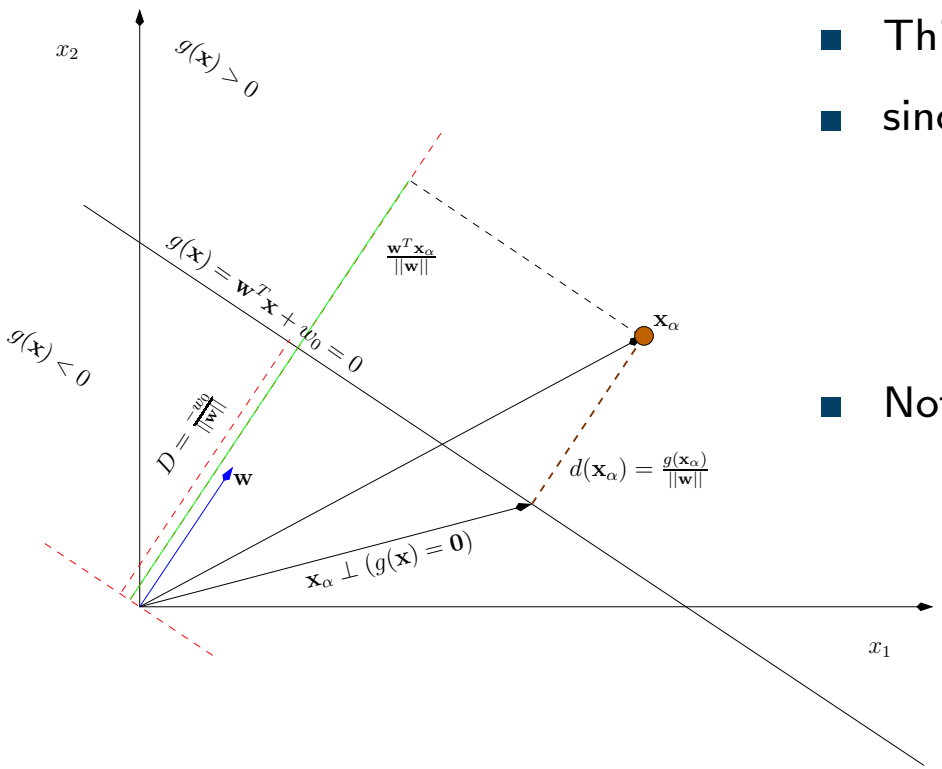
$$\mathbf{x}_{\alpha_\perp} := \mathbf{x}_\alpha \perp (g(\mathbf{x}) = 0)$$

- $\frac{\mathbf{w}}{||\mathbf{w}||}$ is the unit length vector on the direction of $\mathbf{w}$
- obviously

$$\mathbf{x}_\alpha = \mathbf{x}_{\alpha_\perp} + d(\mathbf{x}_\alpha)\frac{\mathbf{w}}{||\mathbf{w}||} \tag{1}$$

- By multiplying eq 1 by $\mathbf{w}^T$ and adding w_0 we can show that $d(\mathbf{x}_\alpha) = \frac{g(\mathbf{x}_\alpha)}{||\mathbf{w}||}$

Distance, D, of the hyperplane $g(\mathbf{x}) = 0$ from the origin of axis, $D = -d(\mathbf{x}_\alpha = 0) = -\frac{w_0}{||\mathbf{w}||}$



- For any $\mathbf{x} \in g(\mathbf{x}) = 0$ its projection on $\frac{\mathbf{w}}{||\mathbf{w}||}$ gives the signed distance, D, of the hyperplane from the origin.
- This projection is given by $\frac{\mathbf{w}^T}{||\mathbf{w}||}\mathbf{x}$,
- since $\mathbf{x} \in g(\mathbf{x}) = 0$ we have

$$D = \frac{\mathbf{w}^T}{||\mathbf{w}||}\mathbf{x} = -\frac{w_0}{||\mathbf{w}||} = -d(\mathbf{x}_\alpha = 0)$$

- Note that:

$$\begin{aligned} d(\mathbf{x}_\alpha) &= \frac{g(\mathbf{x}_\alpha)}{||\mathbf{w}||} \\ &= \frac{\mathbf{w}^T\mathbf{x}_\alpha}{||\mathbf{w}||} + \frac{w_0}{||\mathbf{w}||} \\ &= \frac{\mathbf{w}^T\mathbf{x}_\alpha}{||\mathbf{w}||} - D \end{aligned}$$

Summarizing

For the hyperplane $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0 = 0$ we have:

- $\mathbf{w}$ is the vector normal to the hyperplane and defines its direction
- w_0 defines the position of the hyperplane with respect to the origin of the axes, remember that:
  - $D = -\frac{w_0}{||\mathbf{w}||}$ gives the signed distance of the hyperplane from the origin of the axes.
  - $d(\mathbf{x}_\alpha = \mathbf{0}) = -D$ gives the signed distance of the origin of the axes from the hyperplane.
- For any point $\mathbf{x_a}$,
  - sign$(g(\mathbf{x_a}))$ tells us on which side of the hyperplane $g(\mathbf{x}) = 0$ is $\mathbf{x}_a$.
  - $d(\mathbf{x_a}) = \frac{g(\mathbf{x_a})}{||\mathbf{w}||}$, gives its signed distance from the hyperplane $g(\mathbf{x}) = 0$.

How to compute $g(\mathbf{x}) = 0$

There are many different ways to compute the $\mathbf{w}$, and w_0, needed for the definition of $g(\mathbf{x})$. Just some of them:

- Deriving linear discriminants, and more, using Bayes theorem and assumptions about the distribution governing the data.
- Fisher's linear discriminants
- Perceptron
- Logistic Regression
- Support Vector Machines

LD (and not only) using distributional assumptions I

- Starting from the Bayes theorem we can construct:
  linear or *quadratic*
  discriminant functions
- assuming that instances of each class, c_i, follow a multivariate normal distribution $P(\mathbf{x}|c_i) \sim N(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$.
- From Bayes theorem we have:

$$
\begin{aligned}
\arg\max_{c_i}(P(c_i|\mathbf{x})) &= \arg\max_{c_i} \frac{P(\mathbf{x}|c_i)P(c_i)}{P(\mathbf{x})} \\
&= \arg\max_{c_i} \frac{P(\mathbf{x}|c_i)P(c_i)}{\sum_j P(\mathbf{x}|c_j)P(c_j)} \\
&= \arg\max_{c_i}(P(\mathbf{x}|c_i)P(c_i)) \\
&= \arg\max_{c_i}(\ln(P(\mathbf{x}|c_i)) + \ln(P(c_i)))
\end{aligned}
$$

LD (and not only) using distributional assumptions II

- Since $P(\mathbf{x}|c_i) \sim N(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}_i|^{1/2}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu_i})^T \boldsymbol{\Sigma}_i^{-1}(\mathbf{x}-\boldsymbol{\mu_i})}$
- Then we have the decision rule:

$$\begin{aligned}\arg\max_{c_i}(P(c_i|\mathbf{x})) &= \arg\max_{c_i}[\ln(\frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}_i|^{1/2}}) \\ &\quad -\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^T\boldsymbol{\Sigma}_i^{-1}(\mathbf{x}-\boldsymbol{\mu_i}) \\ &\quad +\ln(P(c_i))]\end{aligned}$$

- Note the presence of the *Mahalanobis* distance, $(\mathbf{x}-\boldsymbol{\mu_i})^T\boldsymbol{\Sigma}_i^{-1}(\mathbf{x}-\boldsymbol{\mu}_i)$, from the class centroid $\boldsymbol{\mu}_i$.
- "All" that is left is estimating $\boldsymbol{\Sigma}_i$s, $\boldsymbol{\mu}_i$s

Learning problem formulation, LD using distributional assumptions

- We have made a number of distributional assumptions
- Now we need to:
  - Define an optimization problem
    - We should define an objective function (based on our distributional assumptions)
    - which we minimize or maximize depending on the function we choose with respect to its parameters.
  - Solve it, i.e. find the parameters that optimize it.
- This is the general approach we will follow *all* the time.

Objective function I, Normal Linear Discriminants (NLD)

- Concretely
  - We have specified a parametric functional form for the class conditional densities $P(\mathbf{x}|c_i)$ (normal distributions)
  - We will use as objective function the **likelihood** function
  - which we will maximize with respect to its parameters (here $\boldsymbol{\mu_i}, \boldsymbol{\Sigma_i}, \pi$)
  - over a training set comprised of N $\mathbf{x}_i$ instances and their class labels y_i
- So we are following a *maximum likelihood* approach.

Objective function II, Likelihood function, (NLD)

- Since we will use the maximum likelihood estimation...
- we need to write down the likelihood function of our training set.
- Any ideas????

Objective function II, Likelihood function, (NLD)

- Since we will use the maximum likelihood estimation...
- we need to write down the likelihood function of our training set.
- Any ideas????

$$p(\mathbf{X}, \mathbf{y}|\boldsymbol{\theta}) = \prod_{i=1}^{N} p(\mathbf{x}_i, y_i|\boldsymbol{\theta})$$

Objective function III, Likelihood function, the $p(\mathbf{x}_i, y_i)$ terms, NLD

- what do we do with the $p(\mathbf{x}_i, y_i)$? How do we compute them?

Objective function III, Likelihood function, the $p(\mathbf{x}_i, y_i)$ terms, NLD

- what do we do with the $p(\mathbf{x}_i, y_i)$? How do we compute them?
  - Assume we have two classes C_1 and C_2
  - with class labels $y = 1$ and $y = 0$ respectively,
  - and $p(C_1) = \pi$.
- For a data point x_i of class C_1 we have $y_i = 1$ and:

$$p(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = p(\mathbf{x}_i, C_1|\boldsymbol{\theta}) =$$

Objective function III, Likelihood function, the $p(\mathbf{x}_i, y_i)$ terms, NLD

- what do we do with the $p(\mathbf{x}_i, y_i)$? How do we compute them?
  - Assume we have two classes C_1 and C_2
  - with class labels $y = 1$ and $y = 0$ respectively,
  - and $p(C_1) = \pi$.
- For a data point x_i of class C_1 we have $y_i = 1$ and:

$$p(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = p(\mathbf{x}_i, C_1|\boldsymbol{\theta}) = p(C_1)p(\mathbf{x}_i|C_1) = \pi N(\mathbf{x}_i|\boldsymbol{\mu_1}, \boldsymbol{\Sigma_1})$$

- for a data point x_i of class C_2 we have $y_i = 0$ and:

$$p(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = p(\mathbf{x}_i, C_2|\boldsymbol{\theta}) =$$

Objective function III, Likelihood function, the $p(\mathbf{x}_i, y_i)$ terms, NLD

- what do we do with the $p(\mathbf{x}_i, y_i)$? How do we compute them?
  - Assume we have two classes C_1 and C_2
  - with class labels $y = 1$ and $y = 0$ respectively,
  - and $p(C_1) = \pi$.
- For a data point x_i of class C_1 we have $y_i = 1$ and:

$$p(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = p(\mathbf{x}_i, C_1|\boldsymbol{\theta}) = p(C_1)p(\mathbf{x}_i|C_1) = \pi N(\mathbf{x}_i|\boldsymbol{\mu_1}, \boldsymbol{\Sigma_1})$$

- for a data point x_i of class C_2 we have $y_i = 0$ and:

$$p(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = p(\mathbf{x}_i, C_2|\boldsymbol{\theta}) = p(C_2)p(\mathbf{x}_i|C_2) = (1-\pi)N(\mathbf{x}_i|\boldsymbol{\mu_2}, \boldsymbol{\Sigma_2})$$

- $\boldsymbol{\theta} = \{\pi, \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2\}$ the parameters of the likelihood function

Objective function III, Likelihood function, the $p(\mathbf{x}_i, y_i)$ terms, NLD

- How do I write a general function form of $\prod_{i=1}^{N} p(\mathbf{x}_i, y_i)$
- i.e. how do I state that I should choose over

$$p(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = p(\mathbf{x}_i, C_1|\boldsymbol{\theta}) = p(C_1)p(\mathbf{x}_i|C_1) = \pi N(\mathbf{x}_i|\boldsymbol{\mu_1}, \boldsymbol{\Sigma_1})$$

or

$$p(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = p(\mathbf{x}_i, C_2|\boldsymbol{\theta}) = p(C_2)p(\mathbf{x}_i|C_2) = (1-\pi)N(\mathbf{x}_i|\boldsymbol{\mu_2}, \boldsymbol{\Sigma_2})$$

for the ith term of the product????

Objective function III, Likelihood function, the $p(\mathbf{x}_i, y_i)$ terms, NLD

- How do I write a general function form of $\prod_{i=1}^{N} p(\mathbf{x}_i, y_i)$
- i.e. how do I state that I should choose over

$$p(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = p(\mathbf{x}_i, C_1|\boldsymbol{\theta}) = p(C_1)p(\mathbf{x}_i|C_1) = \pi N(\mathbf{x}_i|\boldsymbol{\mu_1}, \boldsymbol{\Sigma_1})$$

or

$$p(\mathbf{x}_i, y_i|\boldsymbol{\theta}) = p(\mathbf{x}_i, C_2|\boldsymbol{\theta}) = p(C_2)p(\mathbf{x}_i|C_2) = (1-\pi)N(\mathbf{x}_i|\boldsymbol{\mu_2}, \boldsymbol{\Sigma_2})$$

for the ith term of the product???? Solution:

$$[\pi N(\mathbf{x}_i|\boldsymbol{\mu}_1, \boldsymbol{\Sigma_1}]^{y_i}[(1-\pi)N(\mathbf{x}_i|\boldsymbol{\mu}_2, \boldsymbol{\Sigma_2}]^{1-y_i}$$

Objective function III, NLD

- then the likelihood function is:

$$p(\mathbf{X}, \mathbf{y}|\pi, \boldsymbol{\mu_1}, \boldsymbol{\mu_2}, \boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2) = \prod_{i=1}^{N} [\pi N(\mathbf{x}_i|\boldsymbol{\mu}_1, \boldsymbol{\Sigma_1}]^{y_i} [(1-\pi) N(\mathbf{x}_i|\boldsymbol{\mu}_2, \boldsymbol{\Sigma_2}]^{1-y_i}$$

  where $\mathbf{y} = (y_1, y_2, \ldots, y_n)^\mathsf{T}$ the class labels of the training instances

- and the optimization problem is:

$$\arg \max_{\pi, \boldsymbol{\mu_1}, \boldsymbol{\mu_2}, \boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2} p(\mathbf{X}, \mathbf{y}|\pi, \boldsymbol{\mu_1}, \boldsymbol{\mu_2}, \boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2)$$

- How will we optimize it???

Objective function III, NLD

- then the likelihood function is:

$$p(\mathbf{X}, \mathbf{y}|\pi, \boldsymbol{\mu_1}, \boldsymbol{\mu_2}, \boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2) = \prod_{i=1}^{N} [\pi N(\mathbf{x}_i|\boldsymbol{\mu}_1, \boldsymbol{\Sigma_1}]^{y_i} [(1-\pi) N(\mathbf{x}_i|\boldsymbol{\mu}_2, \boldsymbol{\Sigma_2}]^{1-y_i}$$

  where $\mathbf{y} = (y_1, y_2, \ldots, y_n)^\mathsf{T}$ the class labels of the training instances

- and the optimization problem is:

$$\arg \max_{\pi, \boldsymbol{\mu_1}, \boldsymbol{\mu_2}, \boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2} p(\mathbf{X}, \mathbf{y}|\pi, \boldsymbol{\mu_1}, \boldsymbol{\mu_2}, \boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2)$$

- How will we optimize it??? by taking partial derivatives and setting them to zero.

Optimizing the likelihood I

- It is much more convinient to maximize the $\log$ of the likelihood

$$\begin{aligned}\ln p(\mathbf{X}, \mathbf{y}|\pi, \boldsymbol{\mu_1}, \boldsymbol{\mu_2}, \boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2) &= \sum_{i=1}^{N} y_i \ln[\pi N(\mathbf{x}_i|\boldsymbol{\mu}_1, \boldsymbol{\Sigma_1})] \\ &+ \sum_{i=1}^{N} (1-y_i) \ln[(1-\pi) N(\mathbf{x}_i|\boldsymbol{\mu}_2, \boldsymbol{\Sigma_2})]\end{aligned}$$

- We will first look to maximize with respect to π,
- Extracting the terms of the likelihood that depend on π the likelihood function will be:

$$\sum_{i=1}^{N} [y_i \ln \pi + (1-y_i) \ln(1-\pi)]$$

Optimizing the likelihood II

- Setting the derivative with respect to π to 0 and rearranging we get

$$\pi = \sum_{i=1}^{N} y_i = N_1/N$$

where N_1 is the number of instances of C_1 class, i.e. just the class prior.

- Continuing with $\boldsymbol{\mu}_1$ we select from the log likelihood only the terms that depend on $\boldsymbol{\mu}_1$:

$$\sum_{i=1}^{N} y_i \ln N(\mathbf{x}_i|\boldsymbol{\mu}_1, \Sigma_1) = -1/2 \sum_{i=1}^{N} y_i (\mathbf{x}_i - \boldsymbol{\mu}_1)^T \boldsymbol{\Sigma}_1^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_1) + const$$

Optimizing the likelihood III

- Getting the derivative with respect to $\boldsymbol{\mu}_1$ and setting to zero we get:

$$\boldsymbol{\mu}_1 = 1/N_1 \sum_{i=1}^{N} y_i \mathbf{x}_i$$

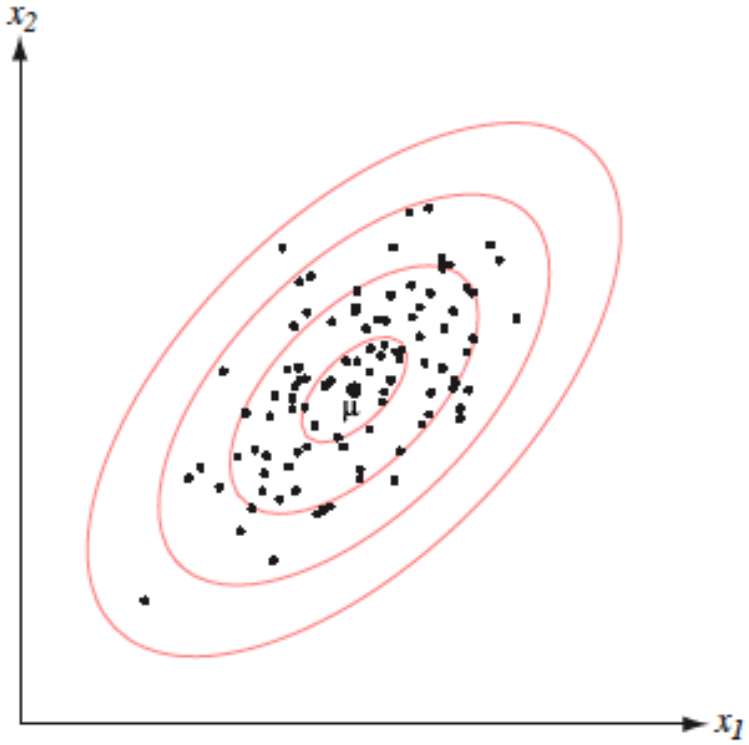
  i.e. just the mean of the C_1 class.
- in the same way for μ_2 we get

$$\boldsymbol{\mu}_2 = 1/(N - N_1) \sum_{i=1}^{N} (1 - y_i) \mathbf{x}_i$$

- In a similar manner we also compute $\boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2$
- Thus the Maximum Likelihood estimates are the class priors, the class conditional means, and the class conditional covariance matrices.

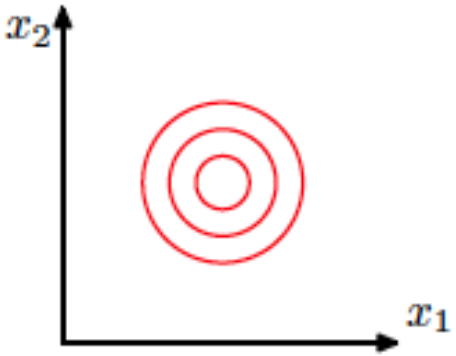
Some notes on the Multivariate Normal Distribution

- $P(\mathbf{x}) \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})}$
- $\boldsymbol{\mu} = (\mu_1, \mu_2, ..., \mu_d)^T$, the d-dimensional mean vector
- $\boldsymbol{\Sigma}$ the $d \times d$ covariance matrix
- the normal distribution is completely specified by $d + d(d+1)/2$ parameters
- the position of the data set is determined by the $\boldsymbol{\mu}$
- the shape of the data set is determined by the $\boldsymbol{\Sigma}$
- points of constant density, i.e. $P(\mathbf{x}) = c$, are hyperellipsoids for which the squared *Mahalanobis* distance $(\mathbf{x} - \boldsymbol{\mu})^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$ is constant.

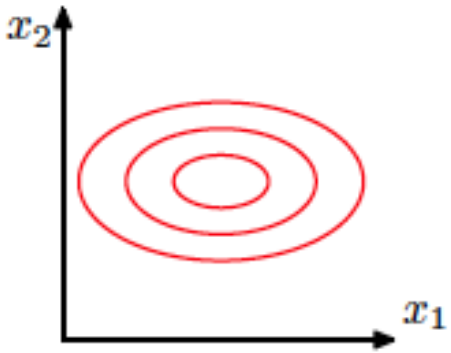


More on the shape of the Multivariate Normal Distribution

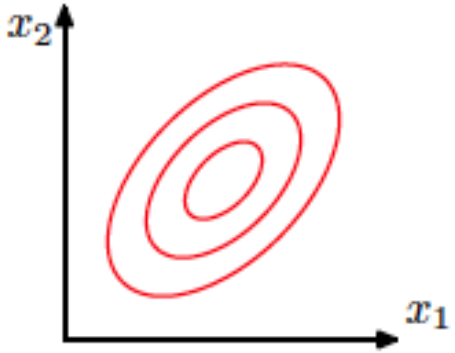
- Different scenarios of the covariance matrix $\mathbf{\Sigma}$.
- Contours represent points of fixed distance from the center $\mathbf{c}$



$\mathbf{\Sigma} = \sigma\mathbf{I}$
contours are circles
circles



$\mathbf{\Sigma}$ is diagonal
contours are
axis-aligned ellipsoids



$\mathbf{\Sigma}$ of general form
contours are
of general form

A bit more on the Multivariate Normal Distribution, MND I

- Remember that the

$$-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}) \tag{2}$$

  Mahalanobis distance is central to the Normal distribution.

- Lets take a moment to examine the general quadratic form:

$$\mathbf{x}^{\mathsf{T}}\mathbf{W}\mathbf{x}$$

  where:

  - first $\mathbf{W}$ is a $d \times d$ full rank symmetric matrix,
  - and then it is a positive semi-definite matrix (PSD), i.e. $\mathbf{x}^T\mathbf{W}\mathbf{x} \geq 0,\ \forall \mathbf{x}$.

A bit more on the Multivariate Normal Distribution, MND II

- Along the way we will examine the properties of the linear transform:

$$\mathbf{A}\mathbf{x},\ \ \mathbf{A} : m \times d, R^d \mapsto R^m$$

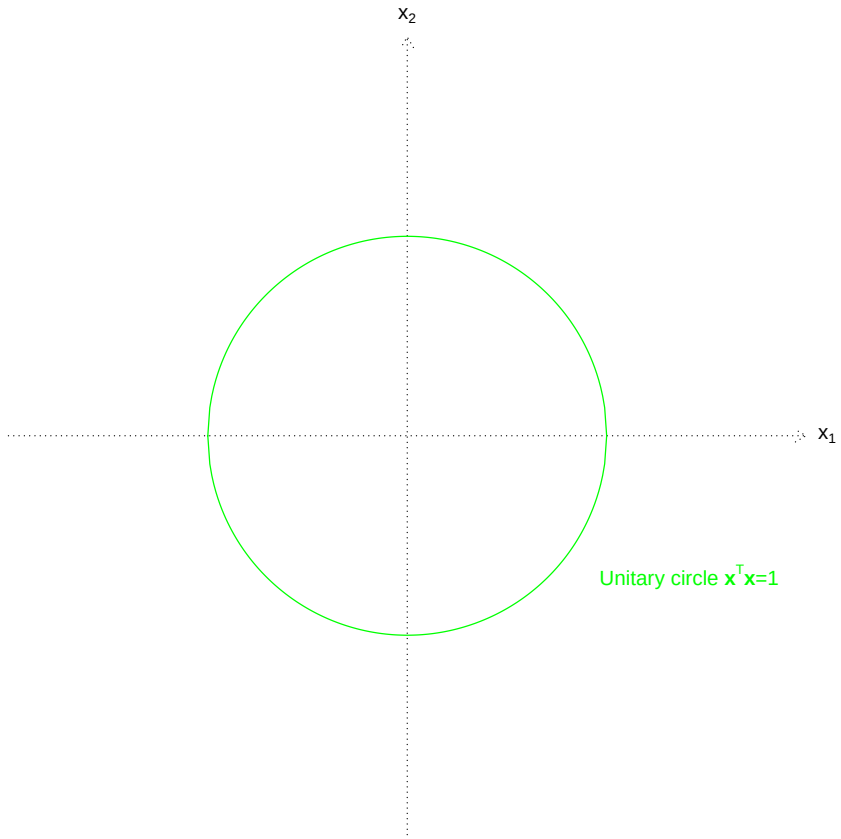
- because the squared norm of the transformed space is:

$$||\mathbf{A}\mathbf{x}||^2 = (\mathbf{A}\mathbf{x})^T\mathbf{A}\mathbf{x} = \mathbf{x}^T\mathbf{A}^T\mathbf{A}\mathbf{x} \overset{\mathbf{W}:=\mathbf{A}}{=} \mathbf{x}^T\mathbf{W}\mathbf{x}$$

- Essentially quadratic forms such as $\mathbf{x}^T\mathbf{W}\mathbf{x}$, where $\mathbf{W}$ can be written as $\mathbf{A}^T\mathbf{A}$ are the squared norms of the $\mathbf{A}\mathbf{x}$ space

Effect of the linear transformation $\mathbf{Ax}$ on the points of the unit circle

- Lets start by exploring the effect of the linear transformation $\mathbf{Ax},\ \mathbf{A} : m \times d, R^d \mapsto R^m$ on the unit circle $\mathbf{x}^T\mathbf{x} = 1$



- demonstration will be done for $d = m$ for simplicity.

Basics on eigen/singular value decomposition

- For square $\mathbf{A}$ of full rank we have the eigen value decomposition

$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^{-1}$$

columns of $\mathbf{U}$ are the eigenvectors of $\mathbf{A}$, $\lambda_i \geq \lambda_{i+1}$.

- For real symmetric $\mathbf{A}$ we have:

$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$$

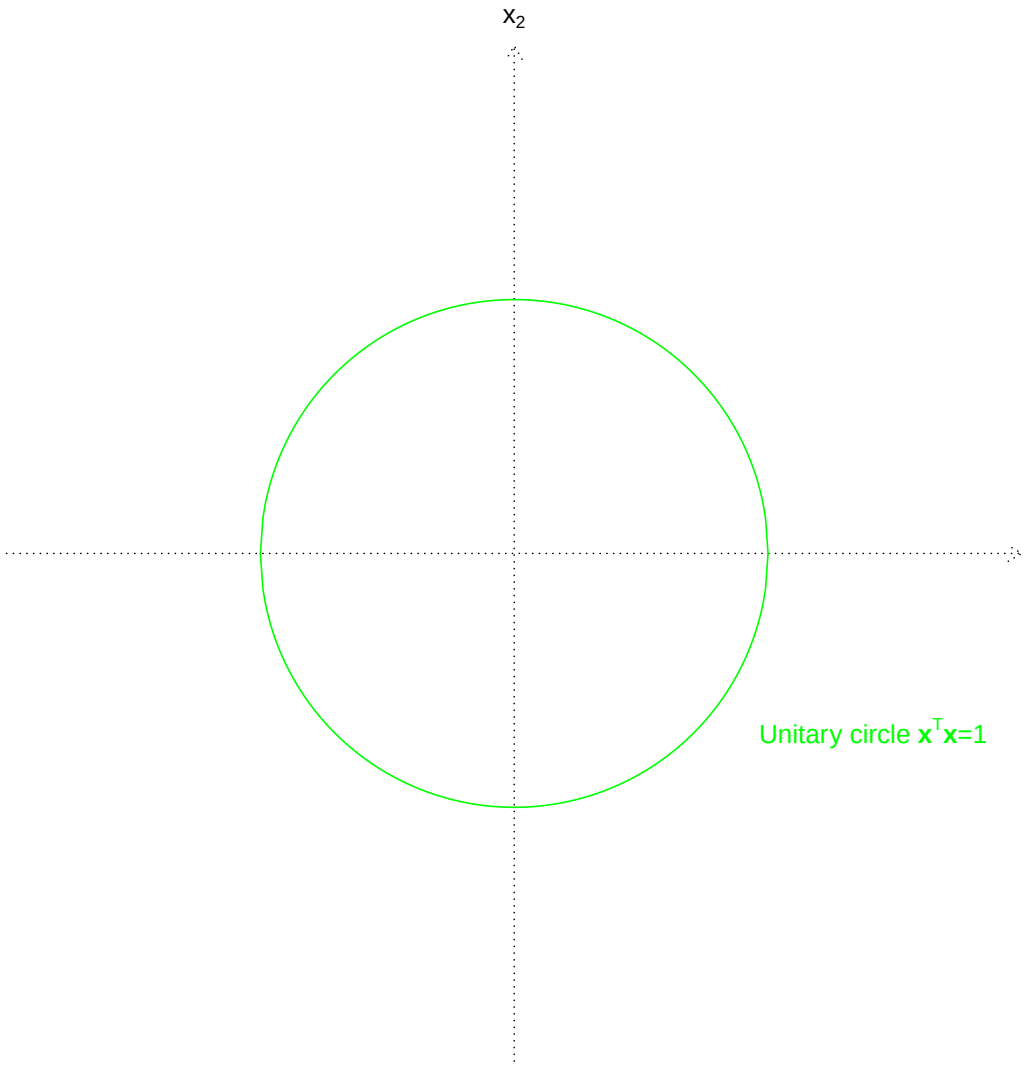
- For rectangular $\mathbf{A}$ we have the singular value decomposition:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T}$$

  - columns of $\mathbf{U} : m \times m$ are eigenvectors of $\mathbf{A}\mathbf{A}^\mathsf{T}$
  - columns of $\mathbf{V} : d \times d$ are eigenvectors of $\mathbf{A}^\mathsf{T}\mathbf{A}$
  - the diagonal element σ_i of the $\mathbf{\Sigma} : m \times d$ matrix is the square root of the λ_i eigenvalue of $\mathbf{A}^T\mathbf{A}$, $\mathbf{A}\mathbf{A}^\mathsf{T}$

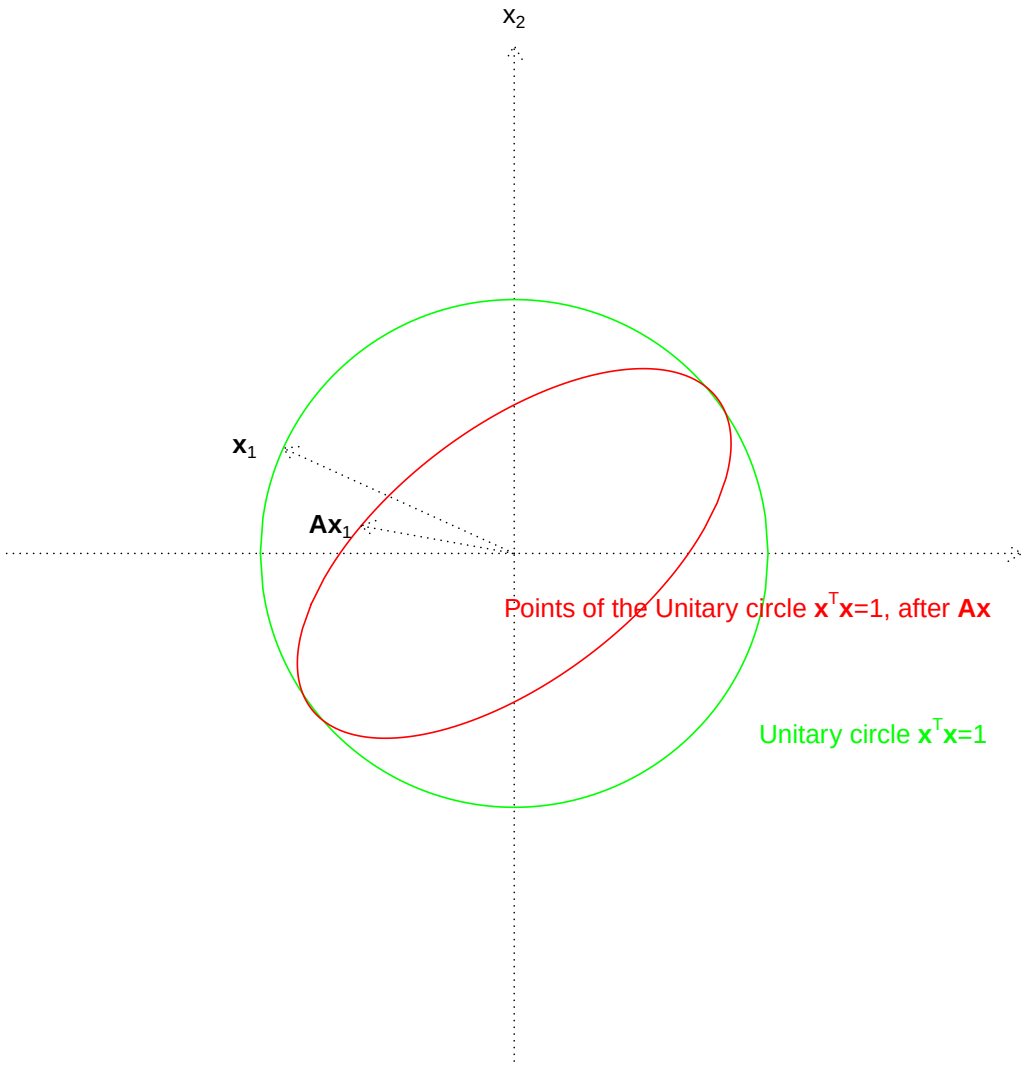
Effect of the linear transformation $\mathbf{Ax}$ on the points of the unit circle

- We will work with the square $\mathbf{A} = \begin{bmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{bmatrix}$
  - that has eigenvectors and eigenvalues:
    - $\mathbf{u}_1 = (0.83, 0.55)^\mathsf{T},\ \lambda_1 = 1$
    - $\mathbf{u}_2 = (-0.71, 0.71),\ \lambda_1 = 0.5$



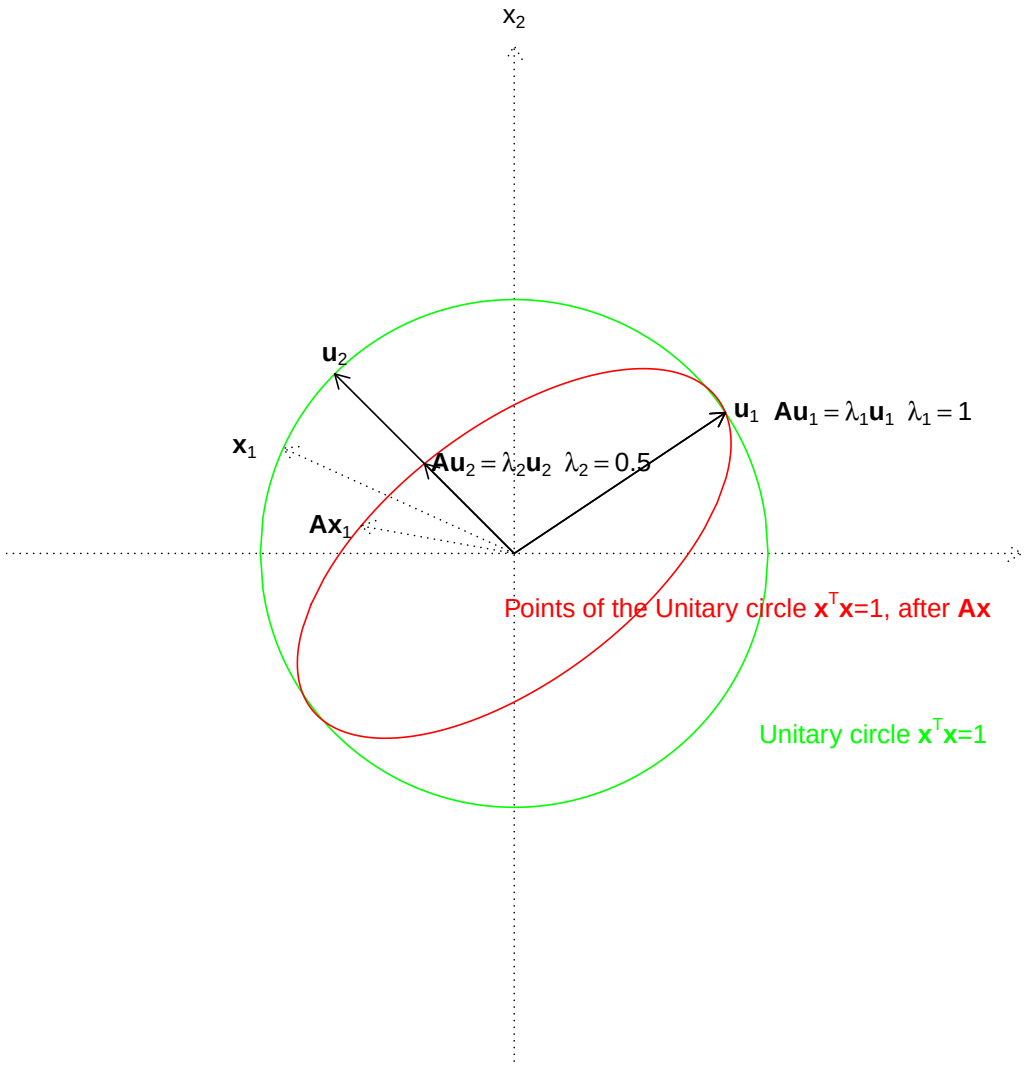
Effect of the linear transformation $\mathbf{Ax}$ on the points of the unit circle

- $\mathbf{Ax}$ transforms the unit circle to an ellipse, squezing or enlarging distances from the origin.
- For any general vector $\mathbf{x}$ the linear transformation $\mathbf{Ax}$ changes its direction
- except for...



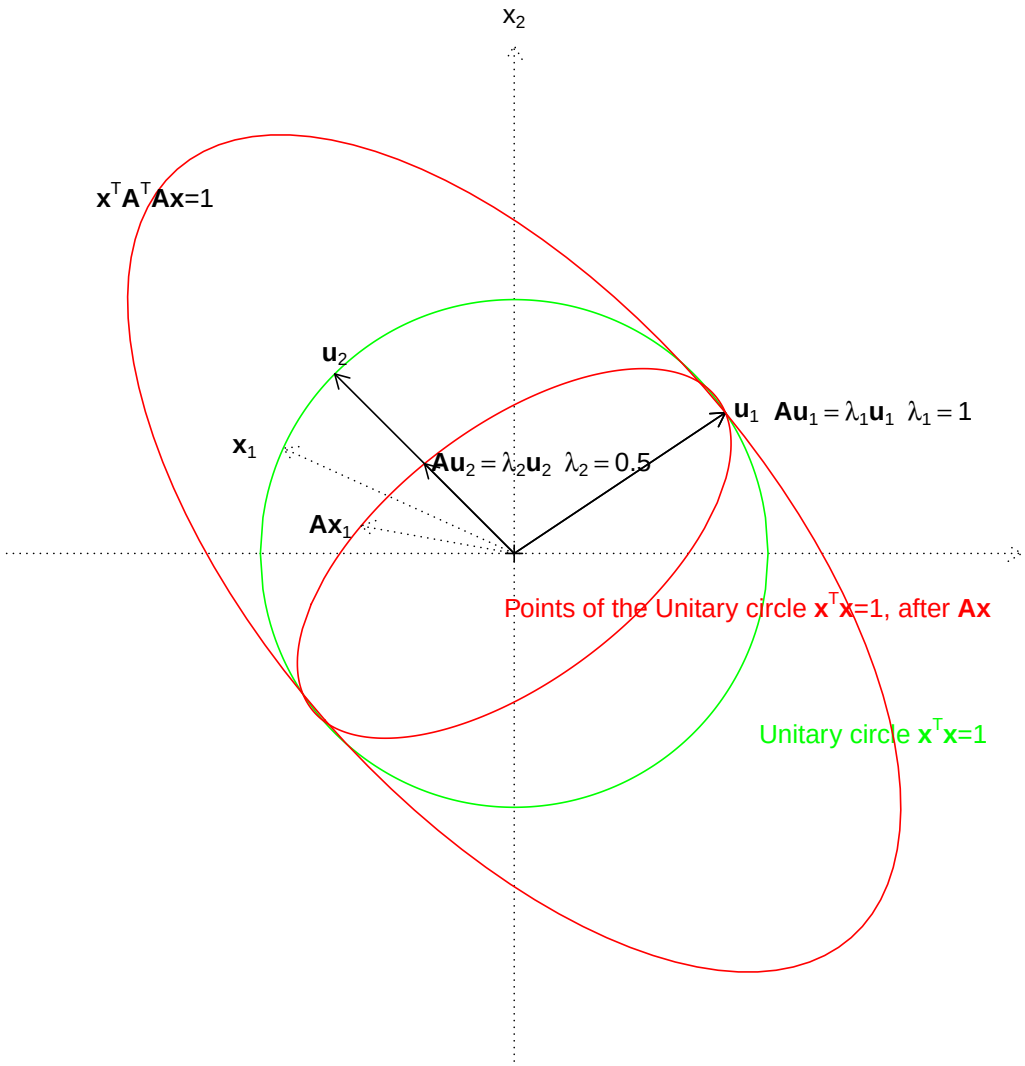
Effect of the linear transformation $\mathbf{Ax}$ on the points of the unit circle

- ... the eigenvectors $\mathbf{u}_1, \mathbf{u}_2$, of $\mathbf{A}$ which are the only vectors whose direction does not change by $\mathbf{A}$.
- They are only scaled by the λ_1 and λ_2 eigenvalues respectively.
- $||\mathbf{Au}_i||^2 = (\mathbf{Au}_i)^\mathsf{T}(\mathbf{Au}_i) = (\lambda_i\mathbf{u}_i)^\mathsf{T}(\lambda_i\mathbf{u}_i) = \lambda_i^2||\mathbf{u}_i||^2$



Effect of the linear transformation $\mathbf{Ax}$ on the points of the unit circle

- Distance from the origin, $||\mathbf{Ax}||$, evolves much slower on the direction of $\mathbf{u}_2$ compared to $\mathbf{u}_1$.



On the $\mathbf{x}^T\mathbf{W}\mathbf{x}$ quadratic form, I

- If $\mathbf{W}$ is *symmetric* matrix $\mathbf{W}$:
  - All its eigenvalues are *real*.
  - Eigenvectors of different eigenvalues are *orthogonal*[1] to each other.
- In addition $\mathbf{W}$ can be decomposed as:

$$\mathbf{W} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T \tag{3}$$

  (spectral theorem), where
  - $\mathbf{V}$ is the orthogonal matrix with columns the orthonormal eigenvectors, $\mathbf{v}_i, i = 1 \ldots d$, of $\mathbf{W}$,
  - and $\mathbf{\Lambda}$ the diagonal eigen values matrix.

On the $\mathbf{x}^T\mathbf{W}\mathbf{x}$ quadratic form, II

- Then:

$$\begin{aligned} \mathbf{x}^T\mathbf{W}\mathbf{x} &= \mathbf{x}^T\mathbf{V}\mathbf{\Lambda}\mathbf{V}^T\mathbf{x} \\ &= [\mathbf{V}^T\mathbf{x}]^T\mathbf{\Lambda}\mathbf{V}^T\mathbf{x} \\ &= \mathbf{y}^T\mathbf{\Lambda}\mathbf{y} \end{aligned} \tag{4}$$

- The linear transformation $\mathbf{y} := \mathbf{V}^T\mathbf{x}$ does ???

On the $\mathbf{x}^T\mathbf{W}\mathbf{x}$ quadratic form, II

- Then:

$$\begin{aligned} \mathbf{x}^T\mathbf{W}\mathbf{x} &= \mathbf{x}^T\mathbf{V}\mathbf{\Lambda}\mathbf{V}^T\mathbf{x} \\ &= [\mathbf{V}^T\mathbf{x}]^T\mathbf{\Lambda}\mathbf{V}^T\mathbf{x} \\ &= \mathbf{y}^T\mathbf{\Lambda}\mathbf{y} \end{aligned} \tag{4}$$

- The linear transformation $\mathbf{y} := \mathbf{V}^T\mathbf{x}$ does ??? a projection of $\mathbf{x}$ to the eigenvector space of $\mathbf{W}$ (a change of basis).
- Moreover:

$$\mathbf{y}^T\mathbf{\Lambda}\mathbf{y} = \sum y_i^2\lambda_i$$

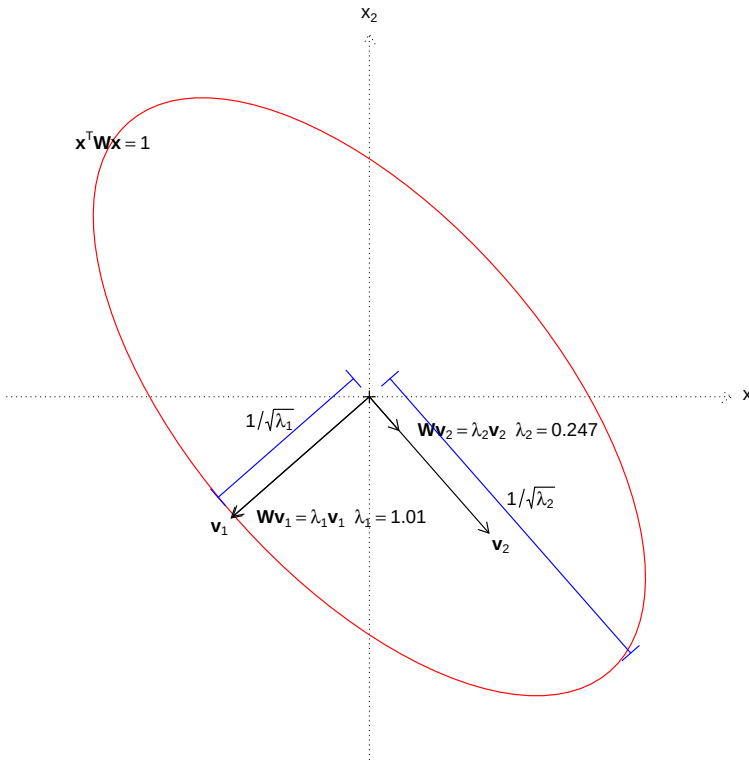
- If in addition $\mathbf{W}$ is PSD, then it has $\lambda_i \geq 0$ and we can write

$$\mathbf{y}^T\mathbf{\Lambda}\mathbf{y} = \sum y_i^2\lambda_i = \sum(\frac{y_i}{1/\sqrt{\lambda_i}})^2 \tag{5}$$

which is...

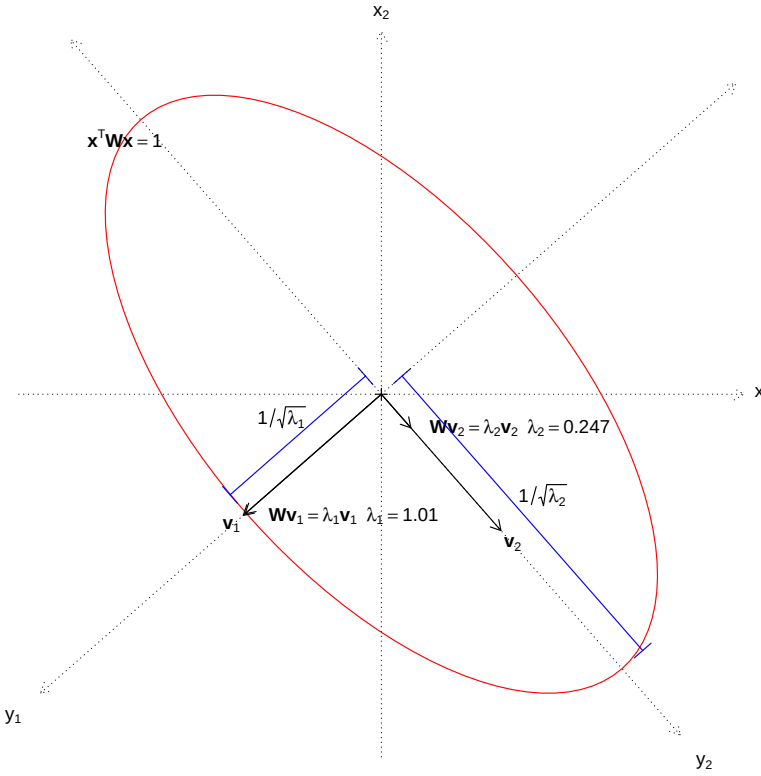
On the $\mathbf{x}^T\mathbf{W}\mathbf{x}$ quadratic form, III

- a hyperelipsoid centered at the origin of the axes:
  - the direction of its main axes is given by the eigenvectors $\mathbf{v}_i$ of $\mathbf{W}$
  - and the scaling of the axes by $\frac{1}{\sqrt{\lambda_i}}$ (λ_i the eigen values of $\mathbf{W}$)
- if $\mathbf{W}$ is only symmetric and not PSD then $\mathbf{y}^T\mathbf{\Lambda}\mathbf{y} = \sum y_i^2\lambda_i$ is a hyperbola.
- (for the figure we set: $\mathbf{W} = \mathbf{A}^T\mathbf{A}$, where $\mathbf{A} = \begin{bmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{bmatrix}$)



On the $\mathbf{x}^T\mathbf{W}\mathbf{x}$ quadratic form, III

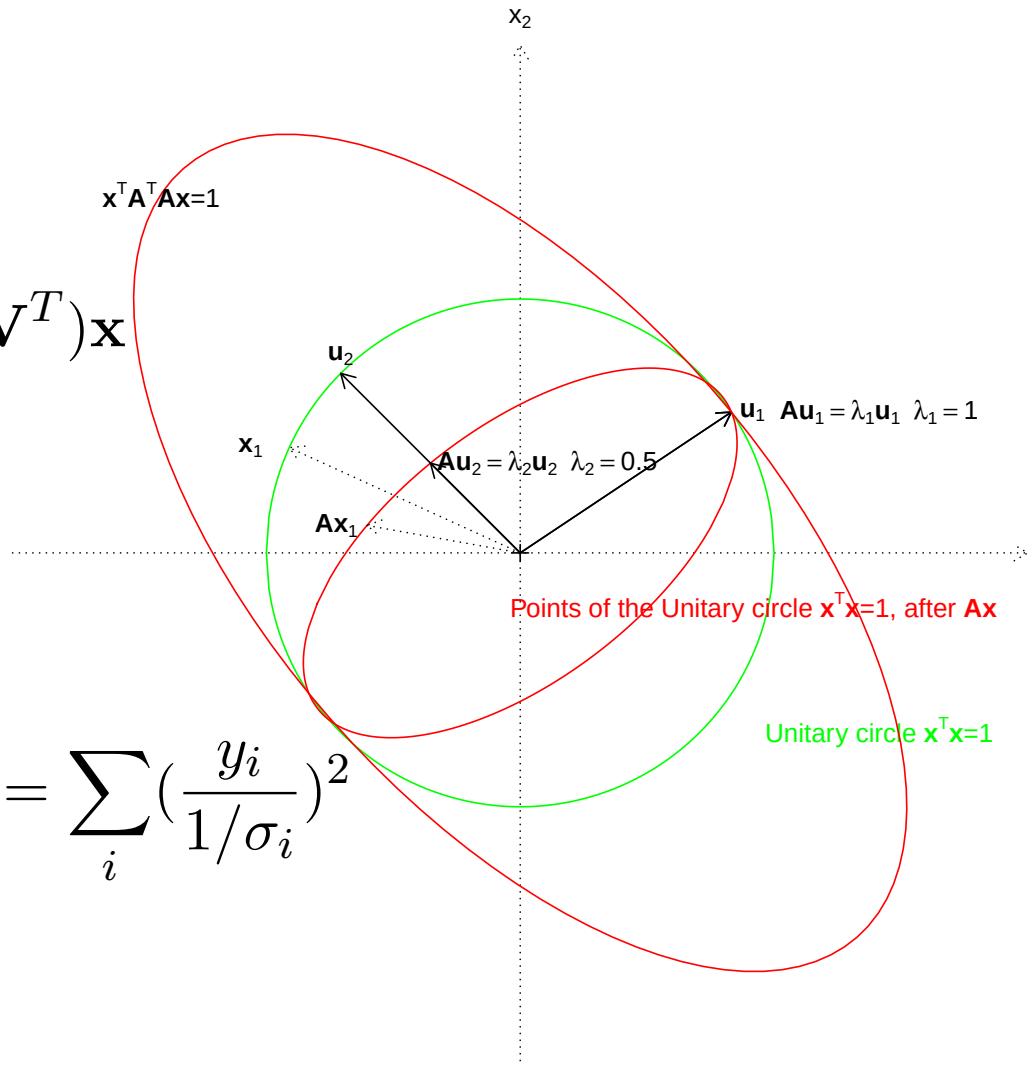
- a hyperelipsoid centered at the origin of the axes:
  - the direction of its main axes is given by the eigenvectors $\mathbf{v}_i$ of $\mathbf{W}$
  - and the scaling of the axes by $\frac{1}{\sqrt{\lambda_i}}$ (λ_i the eigen values of $\mathbf{W}$)
- if $\mathbf{W}$ is only symmetric and not PSD then $\mathbf{y}^T\mathbf{\Lambda}\mathbf{y} = \sum y_i^2\lambda_i$ is a hyperbola.
- (for the figure we set: $\mathbf{W} = \mathbf{A}^T\mathbf{A}$, where $\mathbf{A} = \begin{bmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{bmatrix}$)
- $\mathbf{y} := \mathbf{V}^T\mathbf{x}$ does the projection on the new space given by the main axes of the ellipsoid.



On the relation of $\mathbf{Ax}$ and the $\mathbf{x}^T\mathbf{A}^T\mathbf{Ax}$ quadratic form

- Let $\mathbf{A} = \mathbf{U\Sigma V}^T$ the singular value decomposition of $\mathbf{A}$

$$\begin{aligned}\mathbf{x}^T\mathbf{A}^T\mathbf{Ax} &= \mathbf{x}^T(\mathbf{U\Sigma V}^T)^T(\mathbf{U\Sigma V}^T)\mathbf{x} \\ &= \mathbf{x}^T\mathbf{V}^T\mathbf{\Sigma}^T\mathbf{\Sigma V x}\end{aligned}$$



Let $\mathbf{y} := \mathbf{V}^T\mathbf{x}$ then:

$$\mathbf{x}^T\mathbf{A}^T\mathbf{Ax} = \mathbf{y}^T\mathbf{\Sigma}^T\mathbf{\Sigma y} = \sum_i {y_i}^2\sigma_i^2 = \sum_i (\frac{y_i}{1/\sigma_i})^2$$

- which the same as eq 5 but written with the singular values of $\mathbf{A}$.

On the $(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\mu)$ quadratic form, I

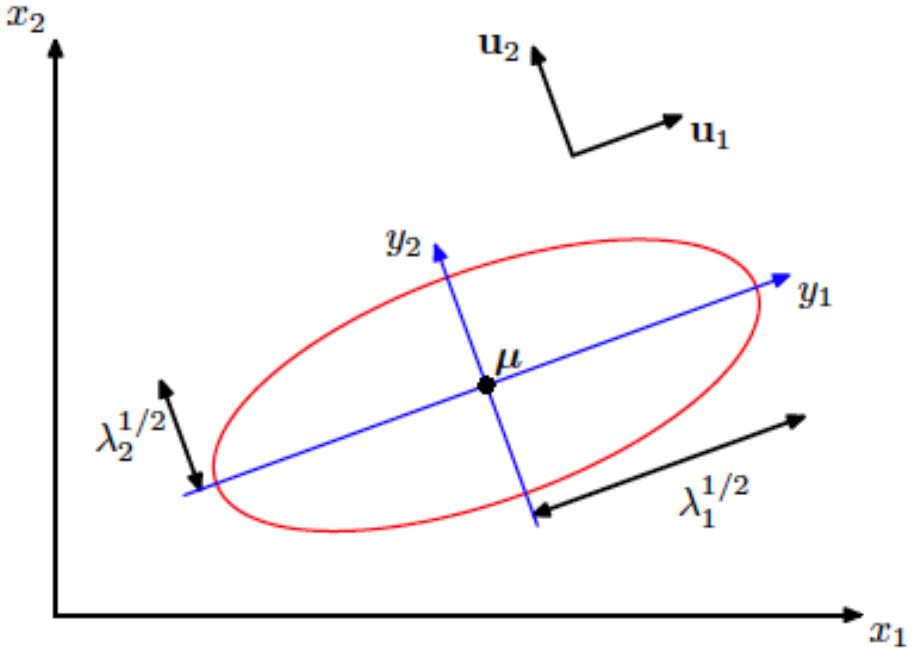
- Let the eigen decomposition of $\boldsymbol{\Sigma} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^T$
- the decomposition of its inverse is $\boldsymbol{\Sigma}^{-1} = \mathbf{U}\boldsymbol{\Lambda}^{-1}\mathbf{U}^T$
- then:

$$\begin{aligned}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\mu) &= (\mathbf{x}-\boldsymbol{\mu})^T\mathbf{U}\boldsymbol{\Lambda}^{-1}\mathbf{U}^T(\mathbf{x}-\boldsymbol{\mu}) \qquad (6)\\ &= [\mathbf{U}^T(\mathbf{x}-\boldsymbol{\mu})]^T\boldsymbol{\Lambda}^{-1}[\mathbf{U}^T(\mathbf{x}-\boldsymbol{\mu})]]\\ &= \mathbf{y}^T\boldsymbol{\Lambda}^{-1}\mathbf{y}\\ &= \sum_i(\frac{y_i}{\sqrt{\lambda_i}})^2\end{aligned}$$

- The transformation $\mathbf{U}^T(\mathbf{x}-\boldsymbol{\mu})$ translates by $\boldsymbol{\mu}$ and projects on the eigenvectors.
- $\sum_i(\frac{y_i}{\sqrt{\lambda_i}})^2$ is...

On the $(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\mu)$ quadratic form, II

- ...



- A hyperelipsoid with:
  - the directions of its main axes given by the eigenvectors, $\mathbf{u}_i$ of the covariance matrix $\boldsymbol{\Sigma}$
  - and the scaling of the axes given by the eigenvalues $\sqrt{\lambda_i}$
- Thus $\mathbf{U}^T(\mathbf{x}-\boldsymbol{\mu})$ translates the origin to $\boldsymbol{\mu}$ and aligns the axes with the main axes of the hyperelipsoid.

Principal component analysis

- Trace of $\mathbf{\Sigma} = \sum_i \sigma_i^2 = \sum_i \lambda_i =$ Total Variance
- In Principal Component Analysis, PCA (a feature transformation method), we project instances on $\mathbf{U}^T\mathbf{x}$.
- Each projection dimension

$$y_i = \mathbf{u}_i^T\mathbf{x}$$

  gives the projection on the ith eigen vector,
- the variance of y_i is λ_i
- the y_is are decorelated
- y_is are ordered by λ_i
- PCA can be used as dimensionality reduction (keep the $\mathbf{u}_i$s with largest λ_i)

PCA and $(\mathbf{x}-\boldsymbol{\mu})^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\mu)$

- We saw that the Mahalanobis distance in the Normal distribution transforms data as:

$$(\mathbf{x}-\boldsymbol{\mu})^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\mu) = [\mathbf{U}^\mathsf{T}(\mathbf{x}-\boldsymbol{\mu})]^\mathsf{T}\boldsymbol{\Lambda}^{-1}[\mathbf{U}^\mathsf{T}(\mathbf{x}-\boldsymbol{\mu})]]$$

$$\mathbf{y} := \mathbf{U}^\mathsf{T}(\mathbf{x}-\boldsymbol{\mu})]^\mathsf{T}\sqrt{\Lambda}^{-1}$$

  i.e. new dimensions y_i have the mean sustracted, are decorelated, and have a variance of 1.

- in PCA the transformation is

$$\mathbf{y} := \mathbf{U}^\mathsf{T}\mathbf{x}$$

  i.e. new dimensions y_i are decorelated and have a variance of λ_i.

Back to LD using distributional assumptions

- Remember we stopped at:

$$\begin{aligned}\arg\max_{c_i}(P(c_i|\mathbf{x})) &= \arg\max_{c_i}[\ln(\frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}_i|^{1/2}}) \\ &\quad -\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^{\mathsf{T}}\boldsymbol{\Sigma}_i^{-1}(\mathbf{x}-\boldsymbol{\mu_i}) \\ &\quad +\ln(P(c_i))]\end{aligned}$$

- Depending on the form of the covariance matrices we get different classifiers.

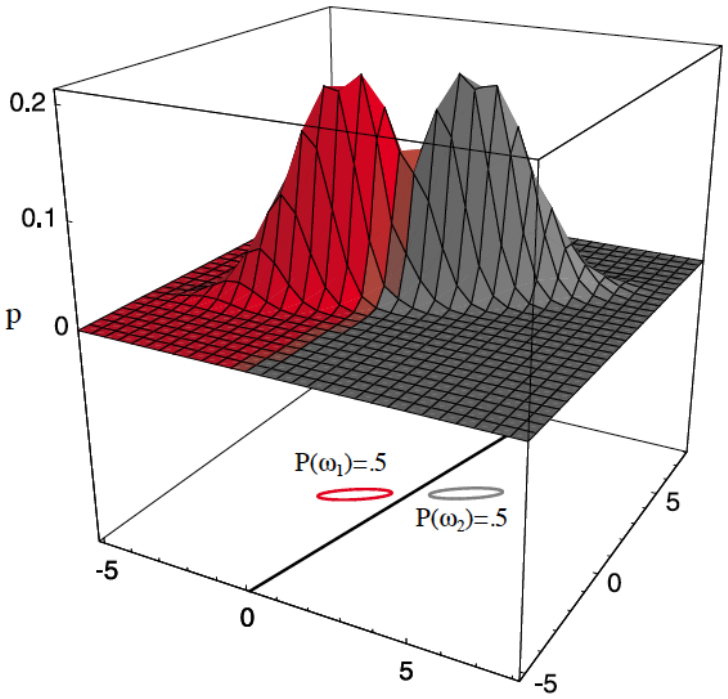
LD using distributional assumptions, $\Sigma_i = \Sigma$

- All classes have the same covariance Σ.
- Then the term, $\ln(\frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}})$, is the same for all classes. So:

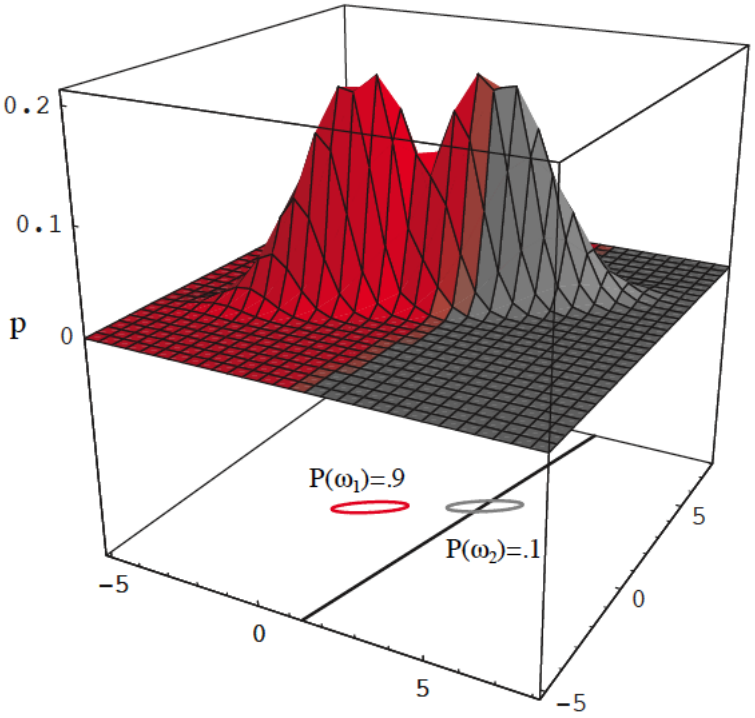
$$\begin{aligned} \arg\max_{c_i}(P(c_i|\mathbf{x})) &= \arg\max_{c_i}[-\frac{1}{2}(\mathbf{x}-\mu_\mathbf{i})^\mathsf{T}\Sigma^{-1}(\mathbf{x}-\mu_\mathbf{i}) \\ &\quad + \ln(P(c_i))] \end{aligned} \tag{7}$$

- So the classification rule is a simple function of the *Mahalanobis* distance from the class centers and the class priors.
- If in equation 7 you develop the Mahalanobis distance you will see that this is a linear function with respect to $\mathbf{x}$, (the quadratic term $\mathbf{x}^t\Sigma^{-1}\mathbf{x}$ is independent of the class and can be omitted).

LD using distributional assumptions, $\Sigma_i = \Sigma$. Examples I

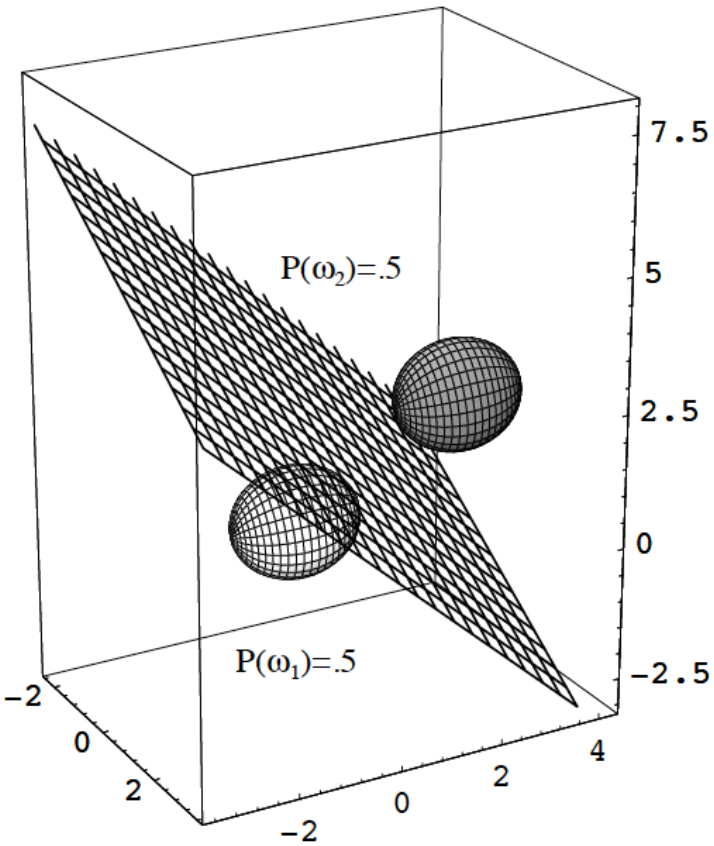


Equal class priors

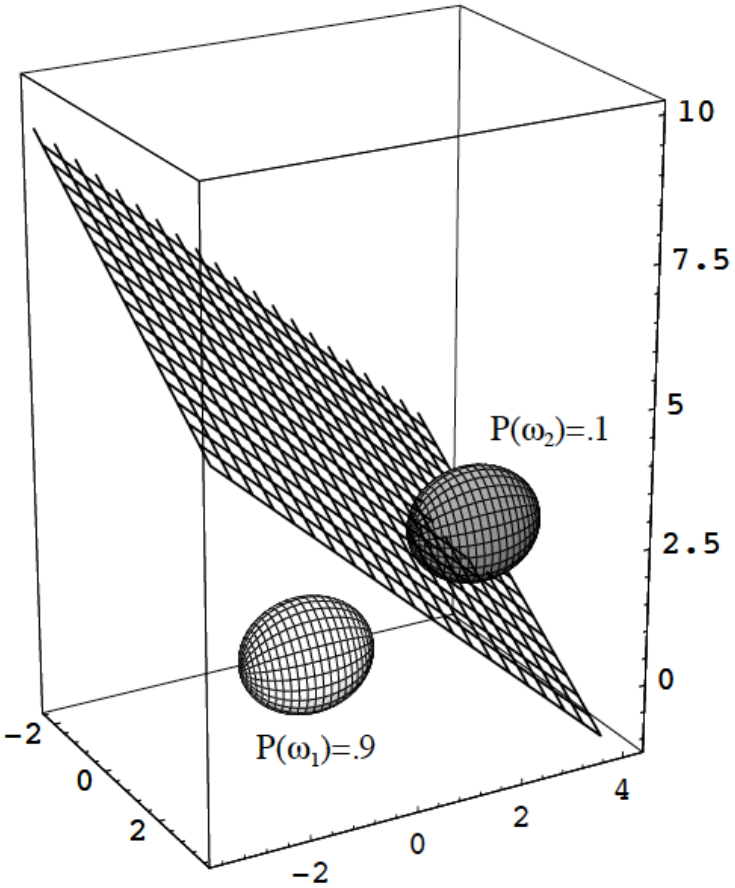


Unequal class priors

LD using distributional assumptions, $\Sigma_i = \Sigma$. Examples II



Equal class priors



Unequal class priors

LD using distributional assumptions, $\mathbf{\Sigma}_i = \sigma^2 \mathbf{I}$

- In this case:

$$-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^t \mathbf{\Sigma}_i^{-1} (\mathbf{x} - \boldsymbol{\mu}_i) =$$

LD using distributional assumptions, $\mathbf{\Sigma}_i = \sigma^2\mathbf{I}$

- In this case:

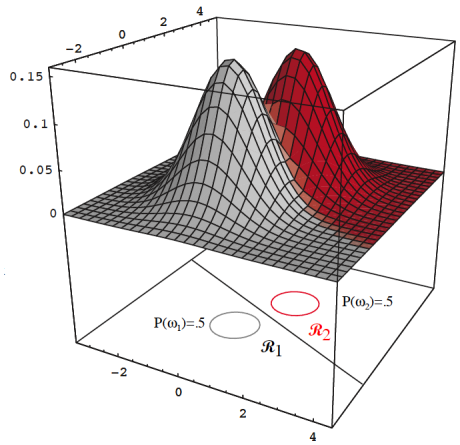
$$-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^t\mathbf{\Sigma}_i^{-1}(\mathbf{x}-\boldsymbol{\mu}_i) = -\frac{||\mathbf{x}-\boldsymbol{\mu}_i||^2}{2\sigma^2}$$

so:

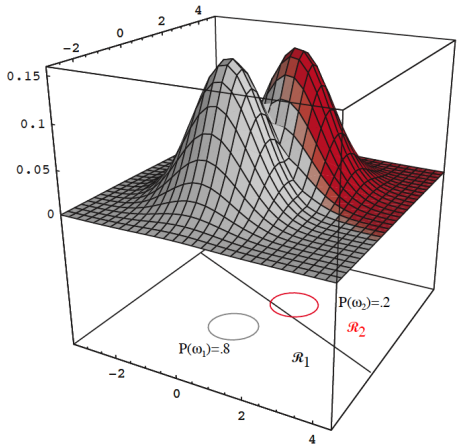
$$\begin{aligned}\arg\max_{c_i}(P(c_i|\mathbf{x})) &= \arg\max_{c_i}[-\frac{||\mathbf{x}-\boldsymbol{\mu}_i||^2}{2\sigma^2} \\ &\quad + \ln(P(c_i))]\end{aligned}$$

- So the classification rule is a simple function of the *Euclidean* distance from the class centers and the class priors.
- Again developing the function we can see that it is linear with respect to $\mathbf{x}$.

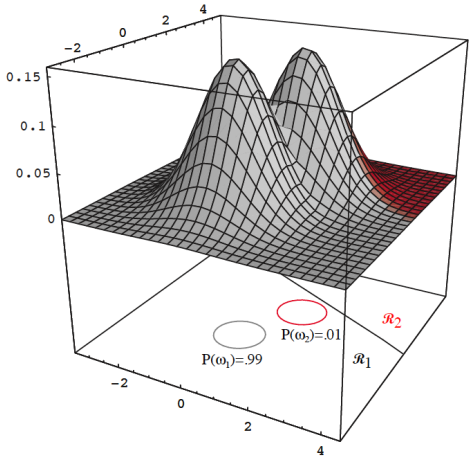
LD using distributional assumptions, $\boldsymbol{\Sigma}_i = \sigma^2 \boldsymbol{I}$



Equal class priors



Unequal class priors



Very unequal class priors

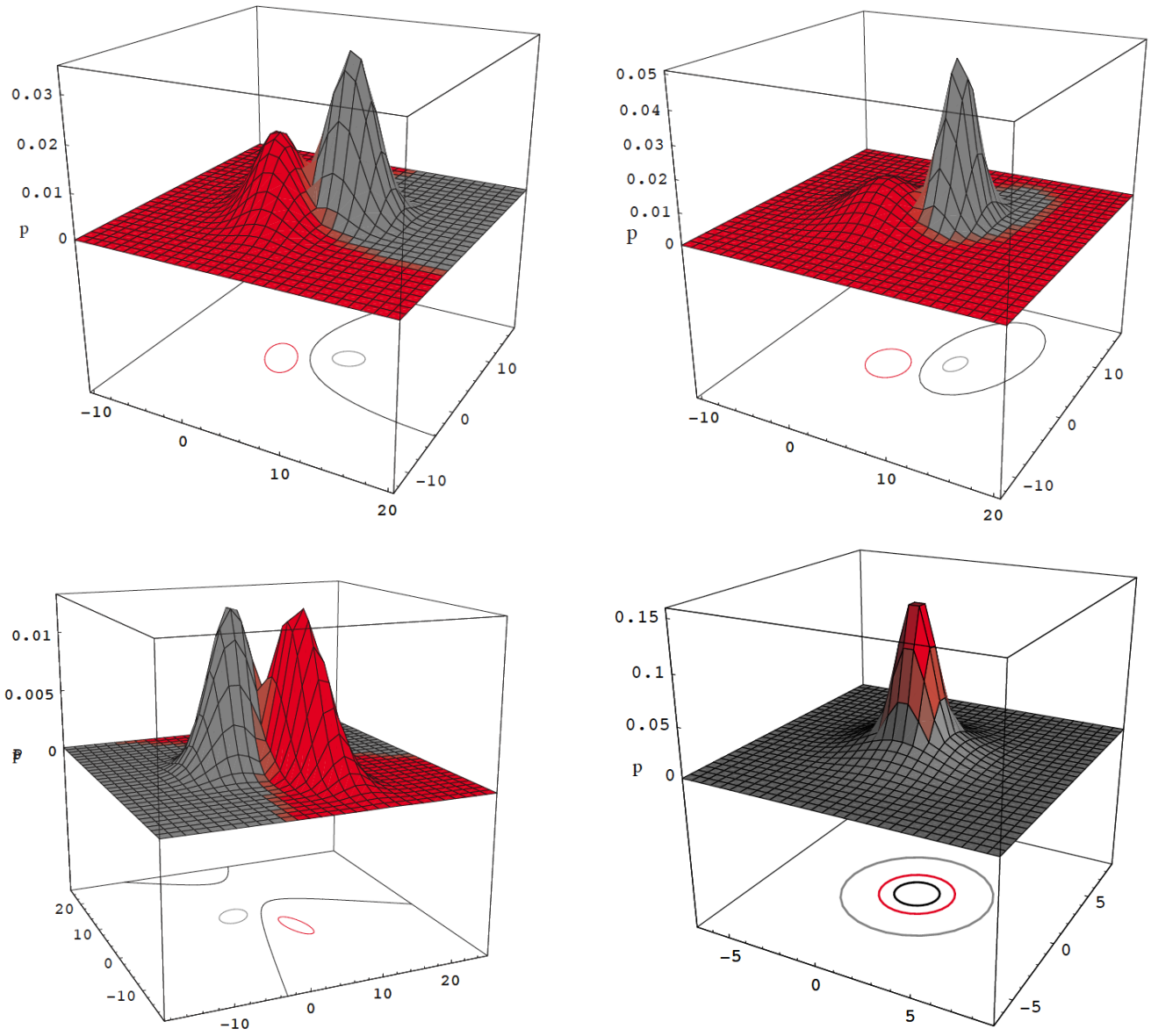
LD using distributional assumptions, arbitrary Σ_i

- In this case no simplifications can be made and the classification rule remains:

$$\begin{aligned}\arg\max_{c_i}(P(c_i|\mathbf{x})) &= \arg\max_{c_i}[\ln(\frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma_i}|^{1/2}}) \\ &\quad -\frac{1}{2}(\mathbf{x}-\mu_\mathbf{i})^t\boldsymbol{\Sigma}_i^{-1}(\mathbf{x}-\mu_\mathbf{i}) \\ &\quad +\ln(P(c_i))]\end{aligned} \tag{8}$$

  which is a *quadratic* function of $\mathbf{x}$.
- The decision surfaces are hyperquadrics, and can assume any of the general forms: hyperplanes, pairs of hyperplanes, hyperspheres, hyperellipsoids, hyperparaboloids, quadric and hyperhyperboloids of various types

LD using distributional assumptions, arbitrary Σ_i

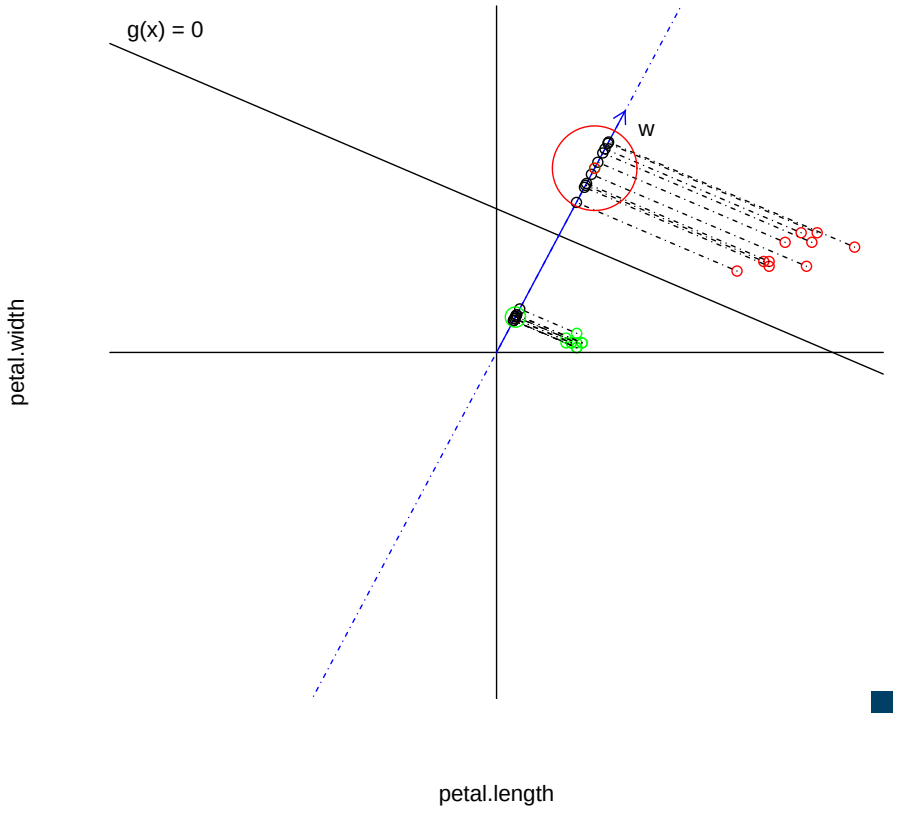


Fisher's Linear Discriminant

Fisher's Linear Discriminant

- Project $\mathbf{x}$ from the initial d-dimensional space on to a single line, i.e., a 1-dimensional space, in case of two classes, or to a $|c| - 1$ dimensional space in case of $|c|$ classes.
- The direction $\mathbf{w}$ of the line should be chosen in such a way so that the separation will be maximum.

Fisher's linear discriminants: Intuition



- Fisher's linear discriminants will look for that projection direction $\mathbf{w}$ that:
  - minimizes the sum of the radius of the two circles, i.e.

$$\min(R_{red} + R_{green})$$

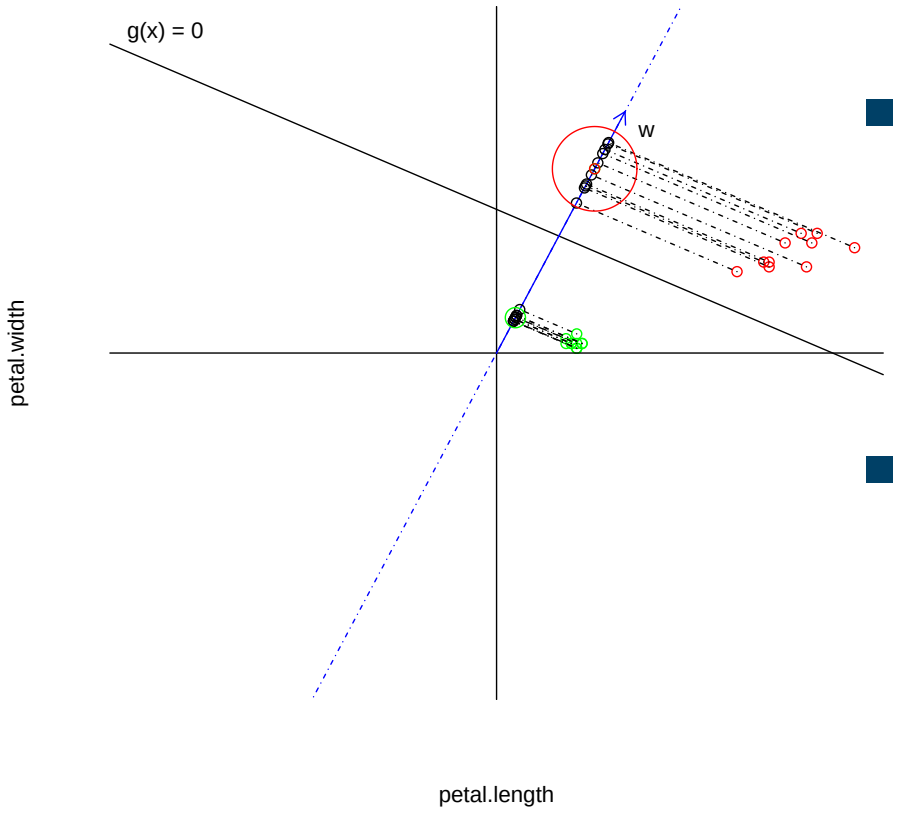
  - places the centers of the two circles, C_{red}, C_{green}, as far as possible from each other, i.e.

$$\max(|C_{red} - C_{green}|)$$

- it achieves the above by looking for the direction $\mathbf{w}$ that maximizes:

$$\frac{|C_{red} - C_{green}|}{R_{red} + R_{green}}$$

Fisher's linear discriminants: Intuition



- How can we define the centers C_{red}, C_{green}?
  - We will call the distance $|C_{red} - C_{green}|$ *between class distance*
- How can we define the radius R_{red}, R_{green}?
  - We will call the $|R_{red} + R_{green}|$ *within class distance*

Fisher's linear discriminants: Intuition I

- We will work in a two-class setting, thus we project to one dimension, thus projections are just scalars.
- Let $\bar{x}$ denote the Fisher projection of $\mathbf{x}$ instance
- and $\bar{\mu}_i$ denote the Fisher projection of the $\boldsymbol{\mu}_i$ mean of the i class.
- We need measures of:
  - of the distances of the classes in the projected space, *between class distance*
  - of the spread of the instances of each class in the projected space, *within class distance*

Fisher's linear discriminants: Between class distance

- We can measure the *between class distance* in the projected space by...
- ...the squared distance of the projections of the means:

$$(\bar{\mu_1} - \bar{\mu_2})^2 \quad =$$

Fisher's linear discriminants: Between class distance

- We can measure the *between class distance* in the projected space by...
- ...the squared distance of the projections of the means:

$$
\begin{aligned}
(\bar{\mu_1} - \bar{\mu_2})^2 &= (\frac{1}{n_1}\sum_{\mathbf{x_i} \in c_1} \mathbf{w}^T \mathbf{x_i} - \frac{1}{n_2}\sum_{\mathbf{x_i} \in c_2} \mathbf{w}^T \mathbf{x_i})^2 \\
&= (\mathbf{w}^T \boldsymbol{\mu}_1 - \mathbf{w}^T \boldsymbol{\mu}_2)^2 \\
&= \mathbf{w}^T (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)[\mathbf{w}^T (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)]^T \\
&= \mathbf{w}^T (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T \mathbf{w} \\
&= \mathbf{w}^T \mathbf{S_B} \mathbf{w}
\end{aligned}
$$

  which is a measure of how far are the two classes.
- $\mathbf{S_B}$ is called the *between class scatter* matrix, $d \times d$, and is of rank one.

Fisher's linear discriminants: Within class distance

- We need measures of the spread of the instances of each class in the projected space, *within class distance*,
- by the variance of the projections around their respective means

$$
\begin{aligned}
\bar{s}_i^2 &= \sum_{\mathbf{x} \in C_i} (\mathbf{w}^T \mathbf{x} - \mathbf{w}^T \boldsymbol{\mu}_i)^2 \\
&= \sum_{\mathbf{x} \in C_i} [\mathbf{w}^T (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T \mathbf{w}] \\
&= \mathbf{w}^T [\sum_{\mathbf{x} \in C_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T] \mathbf{w} \\
&= \mathbf{w}^T \mathbf{S}_i \mathbf{w}
\end{aligned}
$$

- $\mathbf{S}_i$ is the *within class scatter* matrix of the i class, and what else????

Fisher's linear discriminants: Within class distance

- We need measures of the spread of the instances of each class in the projected space, *within class distance*,
- by the variance of the projections around their respective means

$$
\begin{aligned}
\bar{s}_i^2 &= \sum_{\mathbf{x} \in C_i} (\mathbf{w}^T \mathbf{x} - \mathbf{w}^T \boldsymbol{\mu}_i)^2 \\
&= \sum_{\mathbf{x} \in C_i} [\mathbf{w}^T (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T \mathbf{w}] \\
&= \mathbf{w}^T [\sum_{\mathbf{x} \in C_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T] \mathbf{w} \\
&= \mathbf{w}^T \mathbf{S}_i \mathbf{w}
\end{aligned}
$$

- $\mathbf{S}_i$ is the *within class scatter* matrix of the i class, and what else????
- .. it is just $(n_i - 1)\mathbf{C}_i$, where $\mathbf{C}_i$ the covariance matrix of the i class.

Fisher's Linear Discriminant, cost function

- So to achieve the intuitive rule we describe Fisher's linear discriminant $\mathbf{w}$ according to:

$$\begin{aligned}
\arg\max_{\mathbf{w}} J(\mathbf{w}) &= \arg\max_{\mathbf{w}} \frac{(\bar{\mu_1} - \bar{\mu_2})^2}{\bar{s}_1^2 + \bar{s}_2^2} \\
&= \arg\max_{\mathbf{w}} \frac{\mathbf{w}^T \mathbf{S_B} \mathbf{w}}{\mathbf{w}^T (\mathbf{S}_1 + \mathbf{S}_2) \mathbf{w}} \\
&= \arg\max_{\mathbf{w}} \frac{\mathbf{w}^T \mathbf{S_B} \mathbf{w}}{\mathbf{w}^T \mathbf{S_W} \mathbf{w}} \\
&= \arg\max_{\mathbf{w}} \frac{between\ class\ scatter}{within\ class\ scatter}
\end{aligned} \tag{9}$$

- $\mathbf{S}_W = \mathbf{S}_1 + \mathbf{S}_2$ is the within class scatter matrix.
- note that FLD does not compute w_0 we will see how this can be done.

Optimizing the Fisher's Linear Discriminant cost function I

- One way to maximize $J(\mathbf{w})$ of eq 9 is:
  - differentiate $J(\mathbf{w})$ with respect to $\mathbf{w}$ and set it to zero, by which we get:

$$(\mathbf{w}^T \mathbf{S}_B \mathbf{w})\mathbf{S_W}\mathbf{w} \quad = \quad \mathbf{S_B}\mathbf{w}(\mathbf{w}^T \mathbf{S_W}\mathbf{w})$$

  - We only care about the direction of $\mathbf{w}$ so we can drop the scaling factors $(\mathbf{w}^T \mathbf{S_B}\mathbf{w})$ and $(\mathbf{w}^T \mathbf{S_W}\mathbf{w})$ which gives

$$\mathbf{S_W}\mathbf{w} = \mathbf{S_B}\mathbf{w}$$

  - $\mathbf{S_B}\mathbf{w}$ is always on the direction of $(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$ thus the solution is:

$$\mathbf{w} = \mathbf{S_W}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$$

Optimizing the Fisher's Linear Discriminant cost function II

- another way to solve it is to observe that the quantity:

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S_B} \mathbf{w}}{\mathbf{w}^T \mathbf{S_W} \mathbf{w}}$$

  is known as the *generalized Rayleigh quotient*
- Then the maximizer of $J(\mathbf{w})$ is given by the generalized eigen problem :

$$\mathbf{S_B} \mathbf{w} = \lambda \mathbf{S_W} \mathbf{w}$$

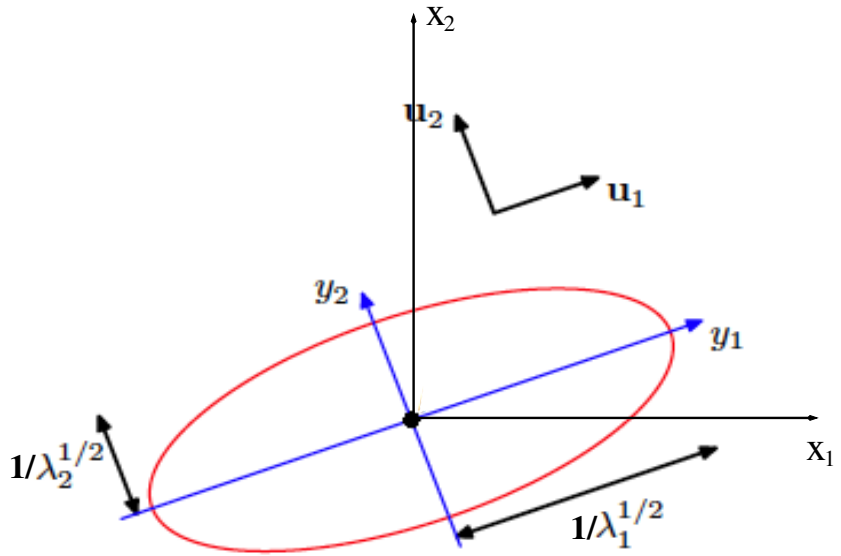
  and we need the maximum eigen vector,
- it can be shown that $\mathbf{w} = \mathbf{S_W}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$.
- In both ways of solving for $\mathbf{w}$ we are still missing w_0.

Fisher's Linear Discriminant, how to choose w_0

- Either you are ready to make some assumptions about the distribution governing your data, in which case you fall back to one of the decision rules given by equations 7,8.
- Either you choose that w_0 which maximizes separation on the training set.

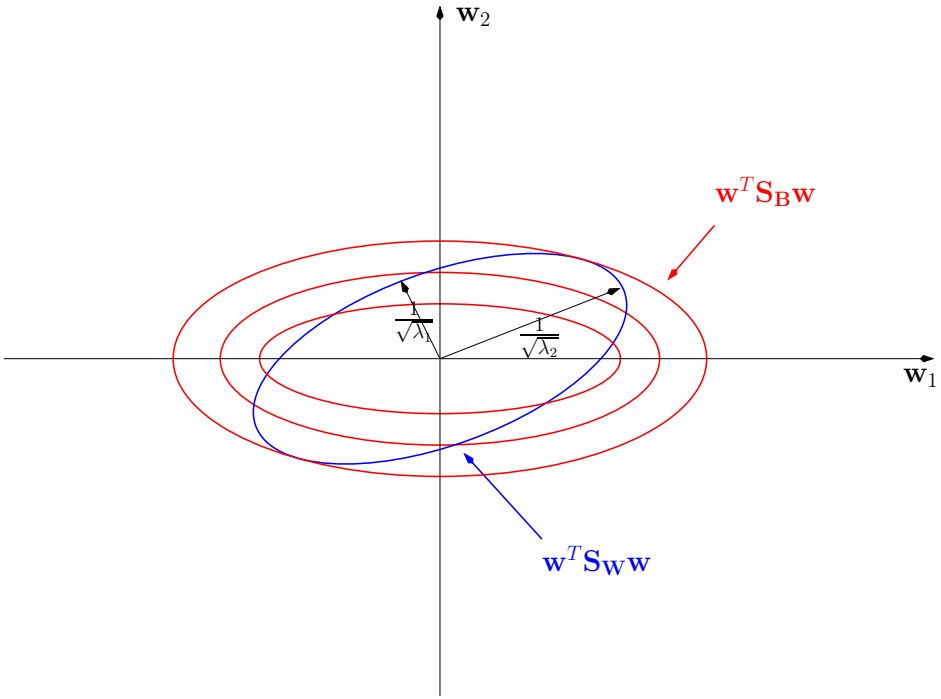
A few words on the Rayleigh quotient I

- The quadratic quantity $\mathbf{x}^T\mathbf{W}\mathbf{x}$,
- when $\mathbf{W} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T \geq 0$, $\mathbf{U} = [\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_d]$ ($\mathbf{u}_i$ column of $\mathbf{U}$)
- is a hyperellipsoid whose direction and length are given by the eigen vectors, $\mathbf{u}_i$, and the eigen values, λ_i, of $\mathbf{W}$ respectively.



A few words on the Rayleigh quotient II

- Thus in the Rayleigh quotient we have two kinds of hyperellipsoids defined by $\mathbf{w}^T\mathbf{S_B}\mathbf{w}$ and $\mathbf{w}^T\mathbf{S_W}\mathbf{w}$



A few words on the Rayleigh quotient III

- Note that:

$$\frac{\mathbf{w}^T\mathbf{S_B}\mathbf{w}}{\mathbf{w}^T\mathbf{S_W}\mathbf{w}} = \frac{\mathbf{w}'^T\mathbf{S_B}\mathbf{w}'}{\mathbf{w}'^T\mathbf{S_W}\mathbf{w}'}, \forall c, \mathbf{w}' = c\mathbf{w}$$

- thus to have a unique solution for $\mathbf{w}$ we need to add a constrain: $\mathbf{w}^T\mathbf{S_W}\mathbf{w} = K$
- and then maximizing the Rayleigh quotient is equivalent to:

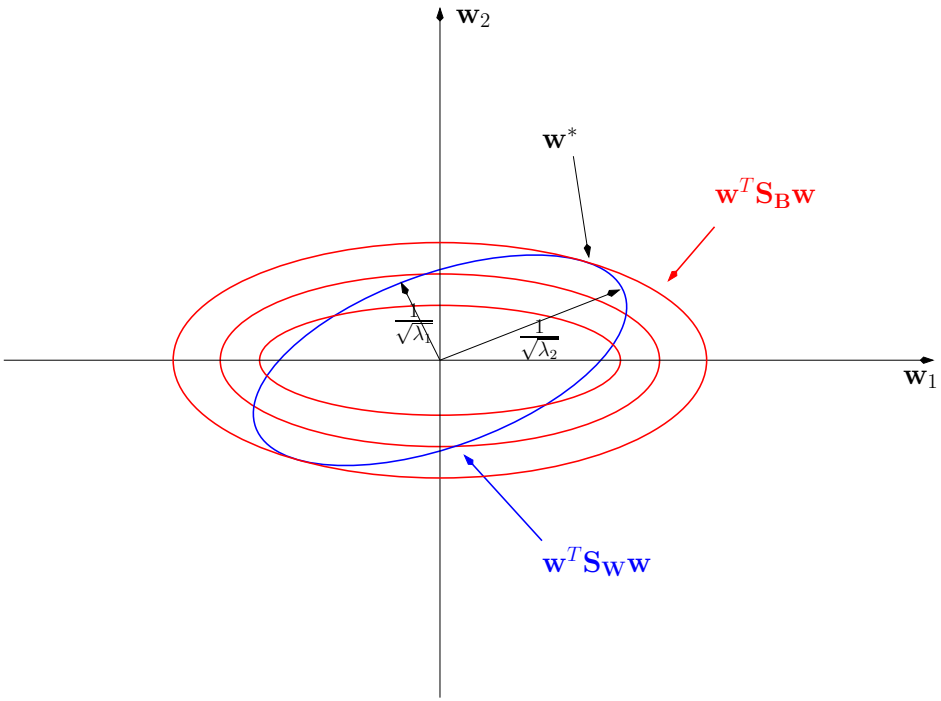
$$\max_{\mathbf{w}} \mathbf{w}^T\mathbf{S_B}\mathbf{w} \text{ subject to } \mathbf{w}^T\mathbf{S_W}\mathbf{w} = K \tag{10}$$

such equations (optimizations under constrains) are solved using the Langrange multipliers. More on the SVM course.

Rayleigh quotient visualization

- the optimal $\mathbf{w}^*$ solution of

$$\max_{\mathbf{w}} \mathbf{w}^T \mathbf{S_B} \mathbf{w} \text{ subject to } \mathbf{w}^T \mathbf{S_W} \mathbf{w} = K \tag{11}$$



is where the outer red circle, $\mathbf{w}^T \mathbf{S_B} \mathbf{w}$, touches the constrain, blue circle, $\mathbf{w}^T \mathbf{S_W} \mathbf{w}$

Perceptron

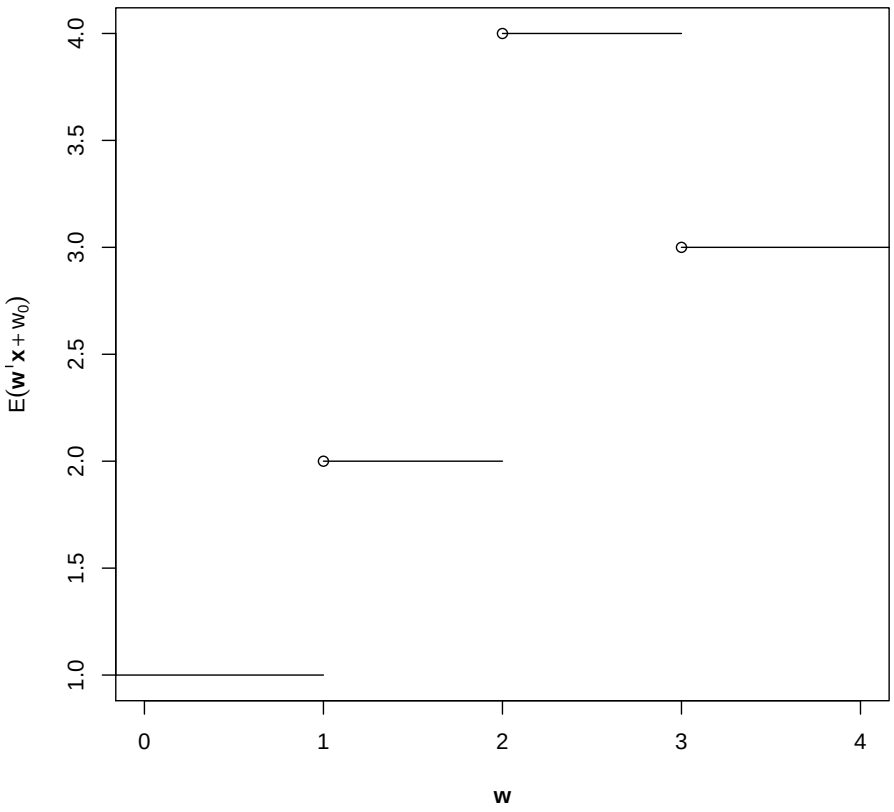
Perceptron

- Linear discriminants based on distributional assumptions
  - follow a probabilistic/generative approach estimating $p(\mathbf{x}, y)$
  - using the likelihood as a cost function to estimate the necessary parameters
- Fisher's linear discriminants
  - follow a geometric approach
  - which leads to the use of the Generalized Rayleigh quotient as the cost function
- Perceptron tries to directly optimize the *missclassification error* (or more precisely a proxy of it).

Perceptron

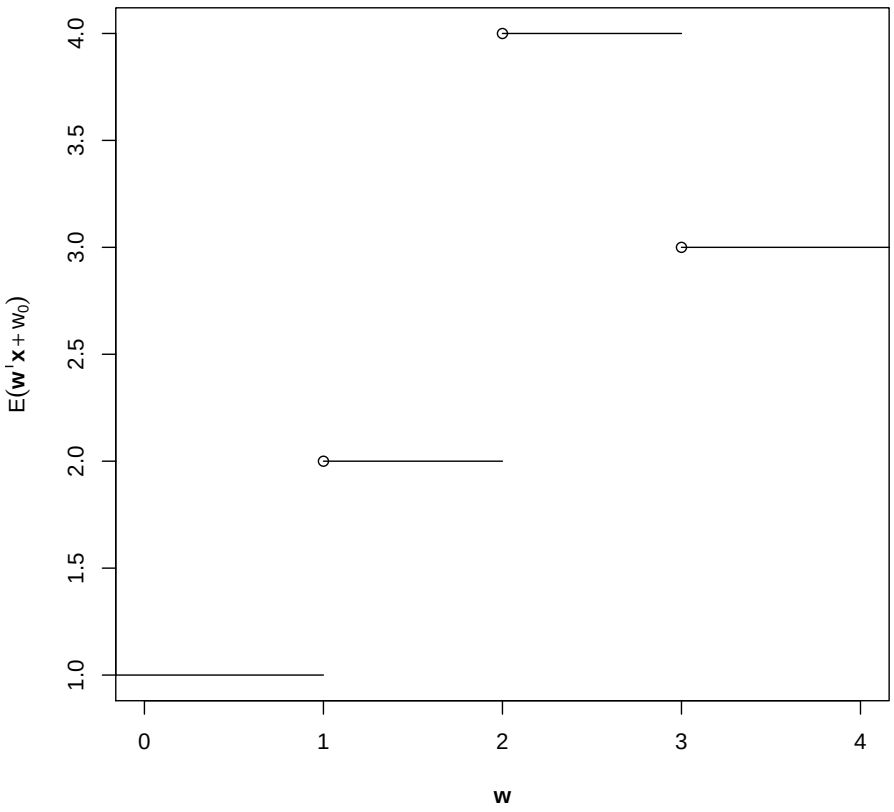
- The Perceptron algorithm provides yet another way to estimate the $\mathbf{w}$, w_0, of the $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0$ function.
- Applicable to two class problems, class encoding $c_1 = 1, c_2 = -1$
- The classification rule it uses is based on $sign(g(\mathbf{x}))$:
  - If $sign(g(\mathbf{x})) = 1$ then c_1
  - If $sign(g(\mathbf{x})) = -1$ then c_2
- Perceptron will find a set of weights *only* if the problem is linearly seperable.

Perceptron Cost function



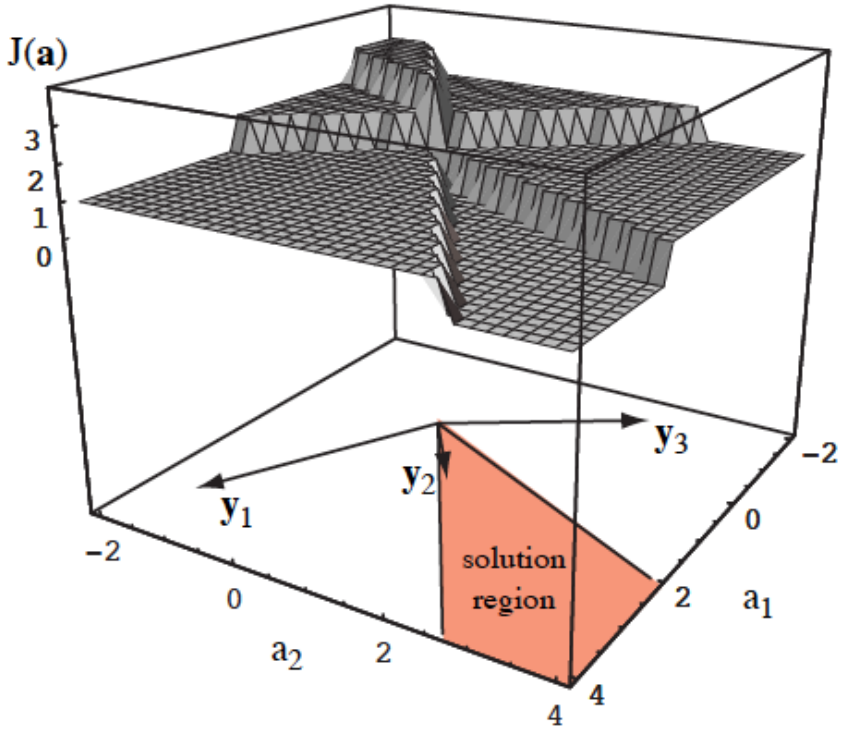
- Missclassification error $E =$ number of instances wrongly classified.
- $\arg\min_{\mathbf{w},w_0} E(\mathbf{w}^T\mathbf{x} + w_0)$
- How would you optimize it?

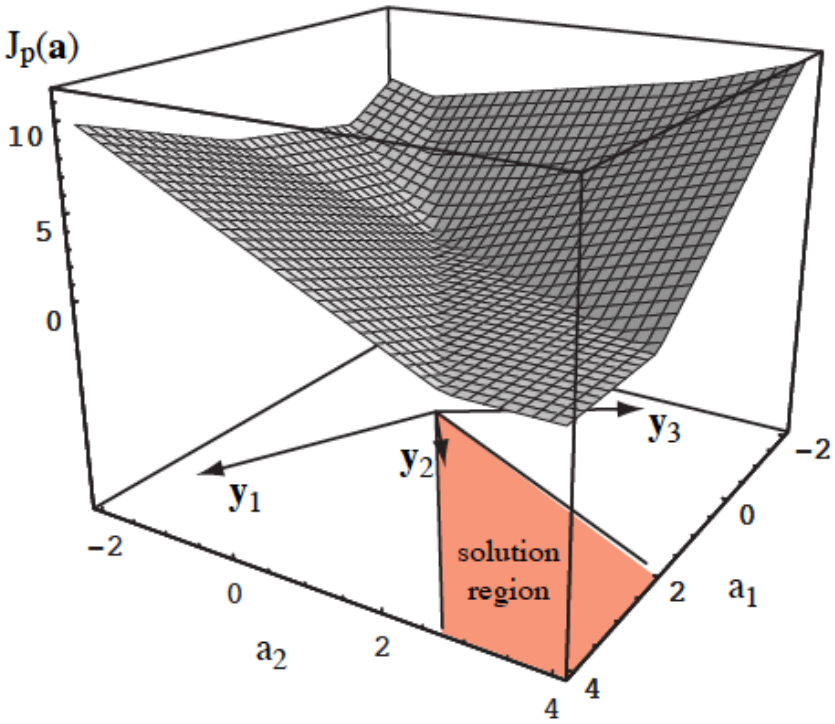
Perceptron Cost function



- Missclassification error $E =$ number of instances wrongly classified.
- $\arg\min_{\mathbf{w},w_0} E(\mathbf{w}^T\mathbf{x} + w_0)$
- How would you optimize it?
- $E(\mathbf{w}^T\mathbf{x} + w_0)$ is piecewise constant with respect to $\mathbf{w}, w_0$
- What is the gradient of this function??? zero almost everywhere...
- Rather problematic for gradient descent...

Examples of Cost functions, [4]





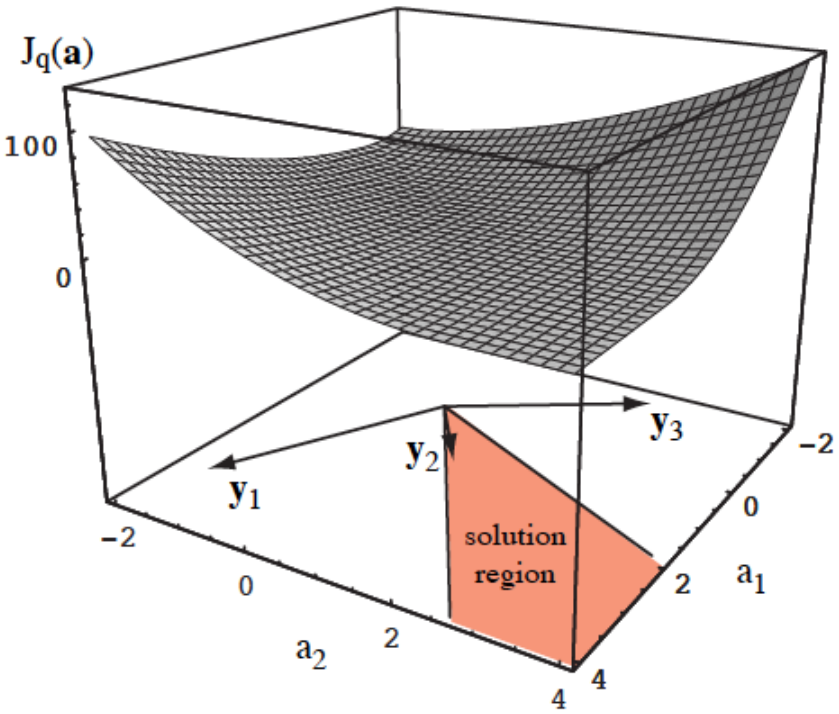
Of wrongly classified instances
Piecewise constant

$-\sum_{\mathbf{x}_i \in M}(\mathbf{w}^T\mathbf{x}_i + w_0)y_i$
Piecewise linear

y_1, y_2, y_3 training samples; a_1, a_2 model parameters; $J(a)$ cost function;

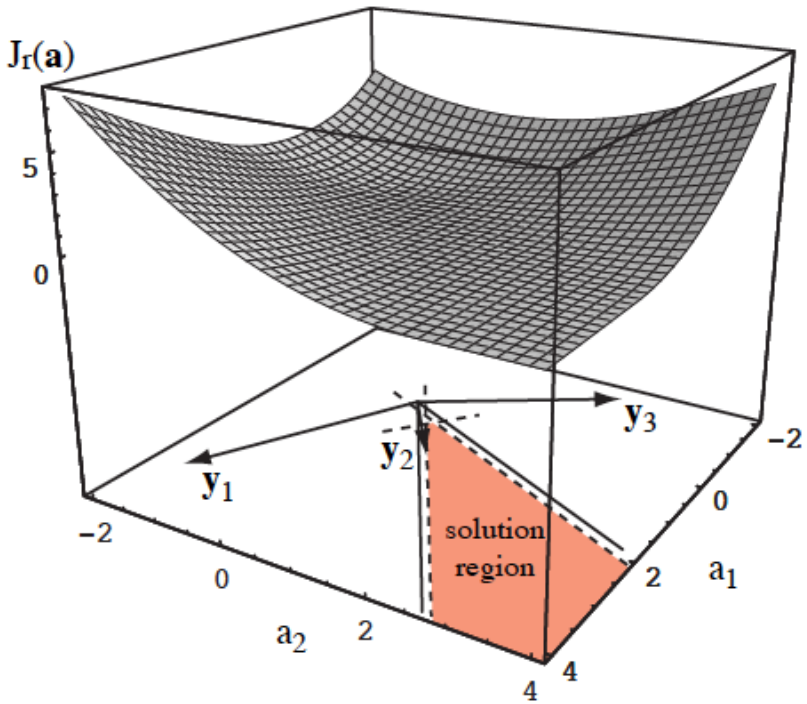
M set of missclassified instances ([4], fig 5.11

Examples of Cost functions



$\sum_{\mathbf{x}_i \in M} (\mathbf{w}^T \mathbf{x}_i + w_0)^2$

Squared Error



$\sum_{\mathbf{x}_i \in M} \frac{((\mathbf{w}^T \mathbf{x}_i + w_0) - b)^2}{||\mathbf{x}_i||^2}$

Squared Error with margin

Perceptron Cost function

- To avoid the problem with piecewise constant error functions...
- ...perceptron uses a proxy for the number of missclassification
- What is a condition that holds for $\mathbf{w}^T\mathbf{x}_i + w_0$ and y_i when the instance $\mathbf{x}_i$ is classified correctly???

Perceptron Cost function

- To avoid the problem with piecewise constant error functions...
- ...perceptron uses a proxy for the number of missclassification
- What is a condition that holds for $\mathbf{w}^T\mathbf{x}_i + w_0$ and y_i when the instance $\mathbf{x}_i$ is classified correctly???

$$(\mathbf{w}^T\mathbf{x_i} + w_0)y_i > 0$$

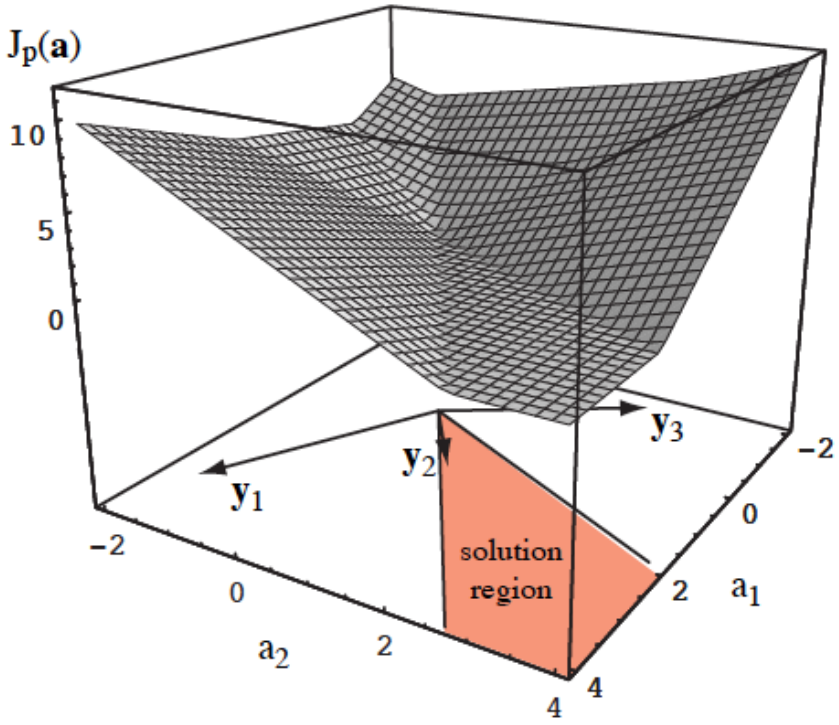
- the cost function of perceptron:
  - assigns zero error to correctly classified instances
  - tries to minimise $-(\mathbf{w}^T\mathbf{x} + w_0)y$ over the missclassified instances.

Perceptron Cost function

- Thus its cost function is given by:

$$E(\mathbf{w}, w_0) = -\sum_{\mathbf{x}_i \in M} (\mathbf{w}^T \mathbf{x}_i + w_0) y_i$$

  where M is the set of missclassified instances.
- which as we said is piecewise linear.
- $E(\mathbf{w}, w_0)$ is proportional to the sum of the distances of the missclassified examples to the decision boundary.
- There is an infinity of solutions orange area of the error graph (if of course the problem is linearly seperable).
- We prefer to have unique well defined solutions.



Optimizing the Perceptron Cost function

- We will minimize the perceptron cost function with gradient descent
- For convenience we will set:

$$\begin{aligned} \mathbf{x}^T &:= (\mathbf{x}^T, 1) \\ \mathbf{w}^T &:= (\mathbf{w}^T, w_0) \end{aligned}$$

  which simplifies:

$$\mathbf{w}^T\mathbf{x} + w_0 \text{ to } \mathbf{w}^T\mathbf{x}$$

- The gradient of

$$E(\mathbf{w}) = -\sum_{\mathbf{x}_i \in M} (\mathbf{w}^T\mathbf{x}_i) y_i$$

  is...

Optimizing the Perceptron Cost function

- We will minimize the perceptron cost function with gradient descent
- For convenience we will set:

$$\begin{aligned} \mathbf{x}^T &:= (\mathbf{x}^T, 1) \\ \mathbf{w}^T &:= (\mathbf{w}^T, w_0) \end{aligned}$$

which simplifies:

$$\mathbf{w}^T\mathbf{x} + w_0 \text{ to } \mathbf{w}^T\mathbf{x}$$

- The gradient of

$$E(\mathbf{w}) = -\sum_{\mathbf{x}_i \in M} (\mathbf{w}^T\mathbf{x}_i) y_i$$

is...

$$\nabla E(\mathbf{w}) = -\sum_{\mathbf{x}_i \in M} \mathbf{x}_i y_i$$

Optimizing the Perceptron Cost function

- How can we write

$$
\begin{aligned}
E(\mathbf{w}) &= -\sum_{\mathbf{x}_i \in M} (\mathbf{w}^T \mathbf{x}_i) y_i \\
\nabla E(\mathbf{w}) &= -\sum_{\mathbf{x}_i \in M} \mathbf{x}_i y_i
\end{aligned}
$$

in matrix form?

Optimizing the Perceptron Cost function

- How can we write

$$
\begin{aligned}
E(\mathbf{w}) &= -\sum_{\mathbf{x}_i \in M} (\mathbf{w}^T \mathbf{x}_i) y_i \\
\nabla E(\mathbf{w}) &= -\sum_{\mathbf{x}_i \in M} \mathbf{x}_i y_i
\end{aligned}
$$

  in matrix form?

$$
\begin{aligned}
E(\mathbf{w}) &= -[\mathbf{X}\mathbf{w}]^{\mathsf{T}} (\mathbf{y} - \mathsf{sign}(\mathbf{X}\mathbf{w})) \\
\nabla E(\mathbf{w}) &= -\mathbf{X}^{\mathsf{T}} (\mathbf{y} - \mathsf{sign}(\mathbf{X}\mathbf{w}))
\end{aligned}
$$

- You should think in terms of matrix and vector operations for computational efficiency.
- Always write your optimization problems and your algorithms in those terms.

Opitimizing the Perceptron Cost function

- Perceptron gradient descent comes into flavors:
  - *batch* the weight vector $\mathbf{w}$ is updated after seeing all the training instances, each such cycle is usually called an *epoch*
  - *single-sample* the weight vector is updated each time we see one instance (can be used to derive an on-line variant of perceptron).

Batch perceptron training

$\eta(.)$ {learning rate, usually a function of the iteration step}
θ {termination criterion}
$k \leftarrow 0$ {iteration step}
$\mathbf{w}^{(k)} \leftarrow random()$
repeat
  $k \leftarrow k + 1$
  $\mathbf{w}^{(k)} \leftarrow \mathbf{w}^{k-1} - \eta(k)\boldsymbol{\nabla} E(\mathbf{w})|_{\mathbf{w}^{(k-1)}} = \mathbf{w}^{(k-1)} + \eta(k) \sum_{\mathbf{x}_i \in M} \mathbf{x}_i y_i$
until $||\mathbf{w}^{(k)} - \mathbf{w}^{(k-1)}||_2 = ||\eta(k) \sum_{\mathbf{x}_i \in M} \mathbf{x}_i y_i||_2 \leq \boldsymbol{\theta}$

Single sample perceptron training

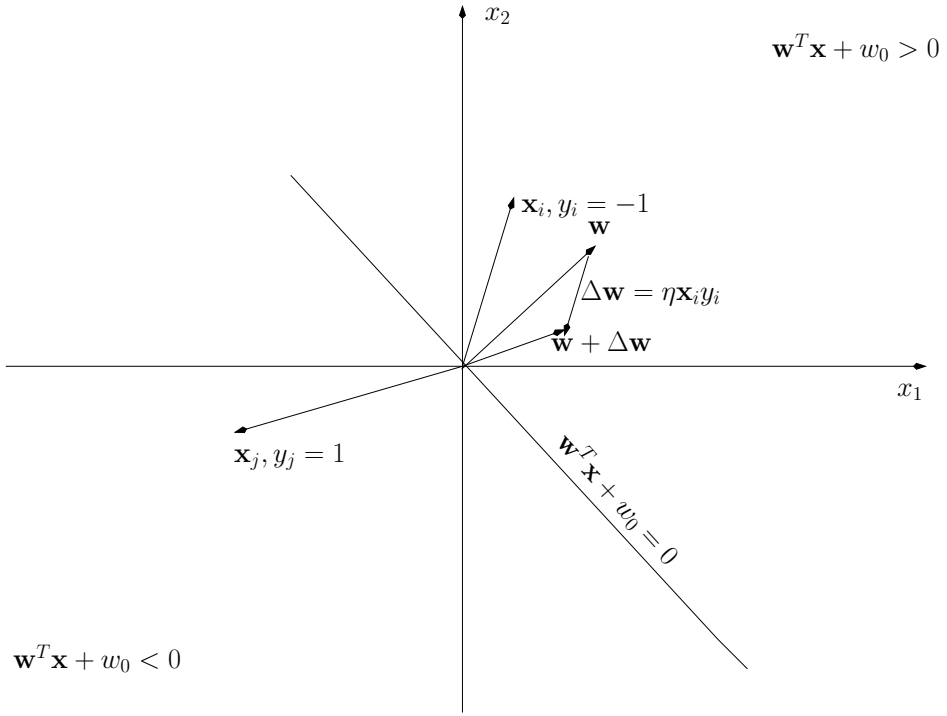
$\eta(.)$ {learning rate, usually a function of the iteration step}
$\boldsymbol{\theta}$ {termination criterion}
$k \leftarrow 0$ {iteration step}
$\mathbf{w} \leftarrow random()$
repeat
  for all $\mathbf{x}_i$ **do**
    if $(\mathbf{w}^T \mathbf{x}_i y_i < 0)$ **then**
      $\mathbf{w} \leftarrow \mathbf{w} + \eta(k)\mathbf{x}_i y_i$
    end if
  end for
until all $\mathbf{x}_i$ correctly classified

Perceptron, weight updating

- Intuitevely when there is a missclassification, i.e. $y_i(\mathbf{w}^{(k)}\mathbf{x}_i) < 0$, of an instance $\mathbf{x_i}$ the weight vector

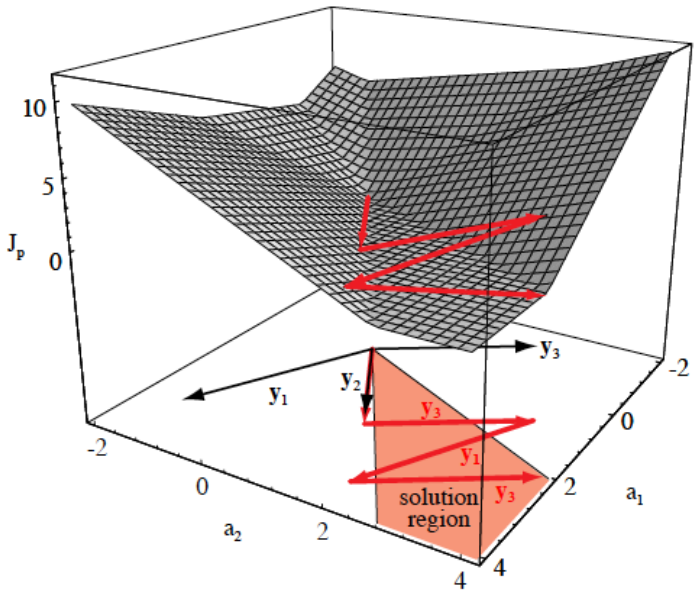
Perceptron, weight updating

- Intuitevely when there is a missclassification, i.e. $y_i(\mathbf{w}^{(k)}\mathbf{x}_i) < 0$, of an instance $\mathbf{x_i}$ the weight vector $\mathbf{w}^{(k)}$ moves on a direction determined by $y_i\mathbf{x}_i$.
- Whenever there is a misclassification the hyperplane moves
  - to the direction of the missclassified example, if it is a positive one
  - oposite of the direction of the missclassified, if it is a negative one



Perceptron, weight updating

- Concecutive weight vectors $\mathbf{w}^{(k)}$ in perceptron update.
- Note how in the second update the weight vector moves further from the solution region.



Controlling the learning rate in (gradient) descent methods

- $\eta(.)$ controls the rate of convergence
  - too small a $\eta(.)$ $\Rightarrow$ too slow convergence
  - too large a $\eta(.)$ $\Rightarrow$ overshooting or even divergence
- Different strategies to control $\eta(.)$ [3, 1]
  - *Heuristic*: start with large η and decrease it with the number of iterations
  - *Exact Line search*: exactly compute η that results in the largest error reduction.
  - *Inexact line search*: compute some η that gives a good error reduction.
    - Backtracking line search

Exact line search

- Main idea: define a one variable, η, optimization problem and solve it at each iteration step of the descent method.

Exact line search

- Let $\Delta\mathbf{w}^{(k)}$ be a descent direction at step k; for example in gradient descent

$$\Delta\mathbf{w}^{(k)} = -\nabla E(\mathbf{w})|_{\mathbf{w}^{(k-1)}}$$

- Define the single variable function:

$$h(\eta) \quad = \quad E(\mathbf{w}^{(k-1)} + \eta\Delta\mathbf{w}^{(k)})$$

to simplify notation we drop the iteration index:

$$h(\eta) = E(\mathbf{w} + \eta\Delta\mathbf{w})$$

- and solve the optimization problem:

$$\eta^{(k)} = \arg\min_{\eta} h(\eta) \tag{12}$$

- Exact line search is used when the cost of solving eq 12 is small compared to cost of establishing the direction [3].

Backtracking line search

- Backtracking line search is an inexact line search in which we look for a good step length η and not necessarily the one that maximily reduces our error function.

Backtracking line search

- $E(\mathbf{w} + \eta \Delta \mathbf{w}) = h(\eta) \sim E(\mathbf{w}) + \eta \nabla E(\mathbf{w})^T \Delta \mathbf{w}$
- For a $\Delta \mathbf{w}$ small enough and given that it is a descent direction then $\nabla E(\mathbf{w})^T \Delta \mathbf{w}$

Backtracking line search

- $E(\mathbf{w} + \eta\Delta\mathbf{w}) = h(\eta) \sim E(\mathbf{w}) + \eta\nabla E(\mathbf{w})^T\Delta\mathbf{w}$
- For a $\Delta\mathbf{w}$ small enough and given that it is a descent direction then $\nabla E(\mathbf{w})^T\Delta\mathbf{w} < 0$
- and given $a \in (0, 0.5)$ we have:

$$\begin{aligned} E(\mathbf{w} + \eta\Delta\mathbf{w}) &= h(\eta) \\ &\sim E(\mathbf{w}) + \eta\nabla E(\mathbf{w})^T\Delta\mathbf{w} \\ &< E(\mathbf{w}) + a\eta\nabla E(\mathbf{w})^T\Delta\mathbf{w} \end{aligned}$$

- A learning rate η is acceptable as long as:

$$E(\mathbf{w} + \eta\Delta\mathbf{w}) \leq E(\mathbf{w}) + a\eta\nabla E(\mathbf{w})^T\Delta\mathbf{w}$$

  otherwise we reduce it to:

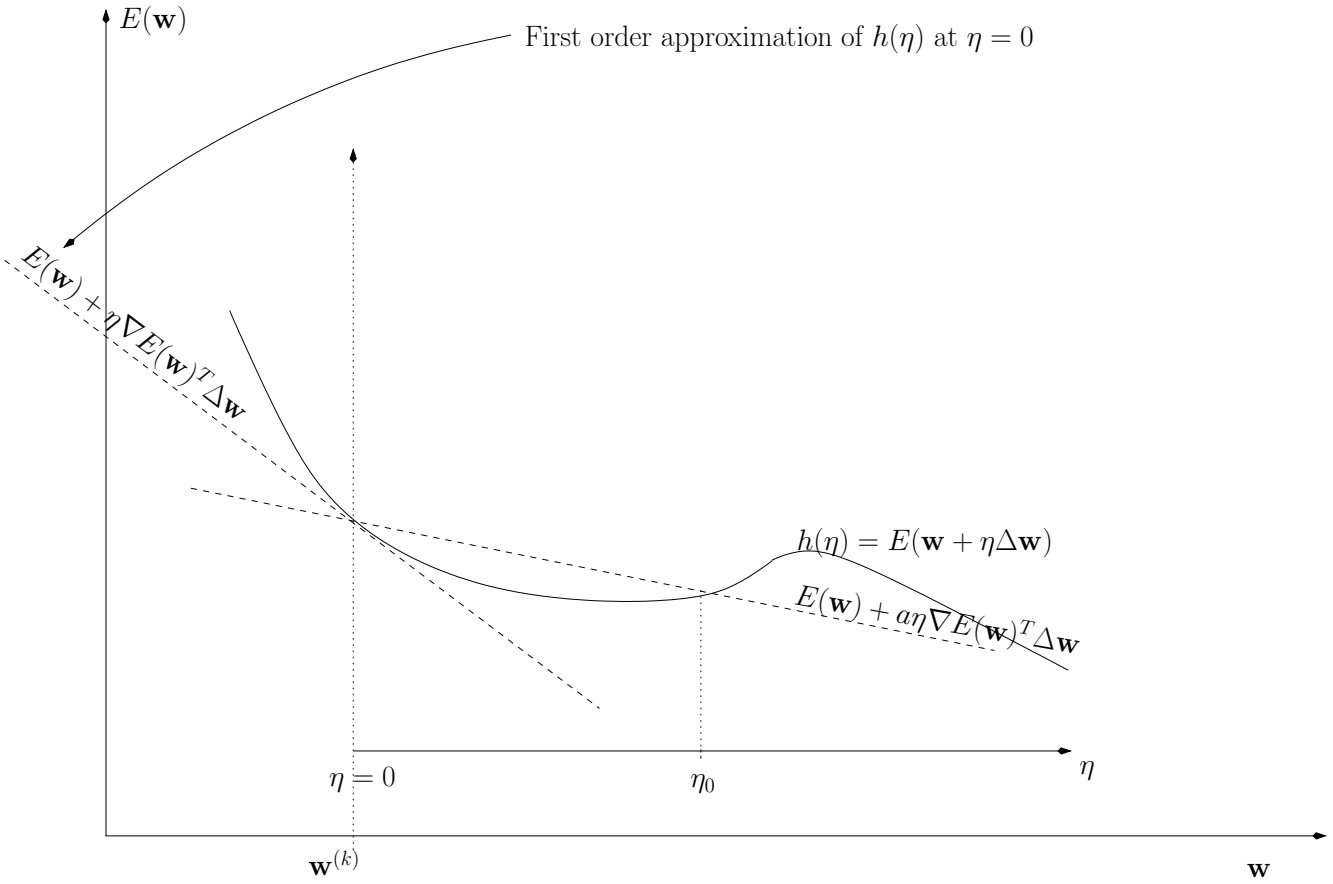
$$\eta \leftarrow \beta\eta$$

  where $\beta \in (0, 1)$

Backtracking line search, Algorithm

$\Delta\mathbf{w}$ {Descent direction, determined for example at each gradient descent step.}
$a \in (0, 0.5)$
$\beta \in (0, 1)$
$\eta := 1$ {Initial learning rate.}
while $E(\mathbf{w} + \eta\Delta\mathbf{w}) > E(\mathbf{w}) + a\eta\nabla E(\mathbf{w})^T\Delta\mathbf{w}$ **do**
  $\eta \leftarrow \beta\eta$
end while

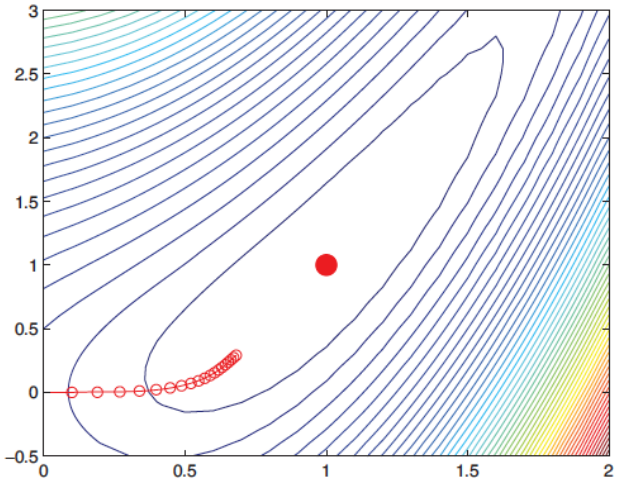
Backtracking line search [3, 1]



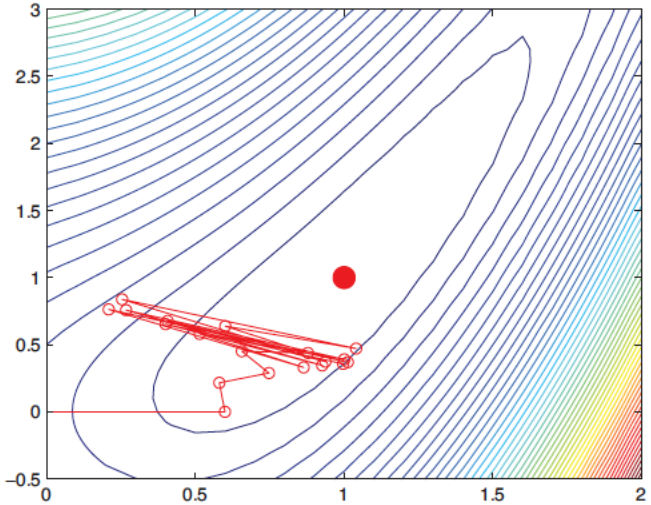
- a is the % of decrease of $E(\mathbf{w} + \eta \Delta \mathbf{w})$ that we accept, with respect to that predicted by its linear approximation $E(\mathbf{w}) + \eta \nabla E(\mathbf{w}) \Delta \mathbf{w}$

Gradient descent examples, [5]

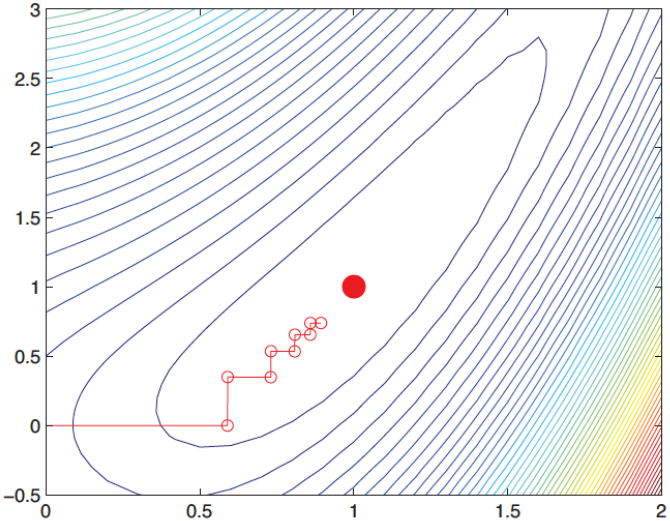
$\eta = 0.1$



$\eta = 0.6$



Exact line search



Logistic regression

Logistic Regression

- In LD with distributional assumptions we used the joint distribution $p(\mathbf{x}, y)$ an approach that is called *generative*.
- Perceptron belongs to the *discriminant function* approaches, which learn directly $g(\mathbf{y}) = y$.
- In logistic regression we will directly estimate $p(y|\mathbf{x})$ an approach that is called *discriminative*.
- Before going to logistic regression we should review the Bernoulli distribution

The Bernoulli distribution

- Consider a random variable y that takes values $\in \{0, 1\}$
- For example y could represent the result of flipping a coin
  - $y = 1$ represents heads
  - $y = 0$ represents tails

  our coin does not need to be fair
- we will denote with μ the probability of $y = 1$, thus:

$$p(y = 1|\mu) = \mu$$

  obviously:

$$p(y = 0|\mu) = 1 - p(y = 1|\mu) = 1 - \mu$$

- obviously:
  - $0 \leq \mu \leq 1$, and
  - $p(y = 0|\mu) = 1 - \mu$

- The probability distribution of y is given by

$$y \sim \text{Bern}(y|\mu) = \mu^y (1-\mu)^{(1-y)} \tag{13}$$

  known as the *Bernoulli distribution*
- Use equation 13 to get the probability of $y = 1$ or $y = 0$. What does it give?
- Its mean and variance are given by

$$\begin{aligned} \mathbb{E}[y] &= \mu \\ \text{var}[y] &= \mu(1-\mu) \end{aligned}$$

Likelihood function of a Bernoulli distribution

- We have a dataset D of n samples drawn i.i.d from that distribution:

$$D = \{y_1, y_2, \ldots y_n\}$$

- Then the *likelihood* of that dataset is:

$$p(D|\mu) =$$

Likelihood function of a Bernoulli distribution

- We have a dataset D of n samples drawn i.i.d from that distribution:

$$D = \{y_1, y_2, \ldots y_n\}$$

- Then the *likelihood* of that dataset is:

$$p(D|\mu) = \prod_{i=1}^{n} p(y_i|\mu) =$$

Likelihood function of a Bernoulli distribution

- We have a dataset D of n samples drawn i.i.d from that distribution:

$$D = \{y_1, y_2, \ldots y_n\}$$

- Then the *likelihood* of that dataset is:

$$p(D|\mu) = \prod_{i=1}^{n} p(y_i|\mu) = \prod_{i=1}^{n} \mu^{y_i}(1-\mu)^{(1-y_i)}$$

- As we said the likelihood is a *function* of the parameters of your distribution/model, what are these parameters here?

Likelihood function of a Bernoulli distribution

- We have a dataset D of n samples drawn i.i.d from that distribution:

$$D = \{y_1, y_2, \ldots y_n\}$$

- Then the *likelihood* of that dataset is:

$$p(D|\mu) = \prod_{i=1}^{n} p(y_i|\mu) = \prod_{i=1}^{n} \mu^{y_i}(1-\mu)^{(1-y_i)}$$

- As we said the likelihood is a *function* of the parameters of your distribution/model, what are these parameters here? μ
- Again as already discussed, different notations can be used for the likelihood function, e.g.

$$l(D; \boldsymbol{\theta}),\ l(\boldsymbol{\theta})$$

$\boldsymbol{\theta}$ the parameters of your distribuition/model (here $\boldsymbol{\theta} = \{\mu\}$).

Log Likelihood function and negative log likelihood

- Given a dataset $D = \{y_1, ..., y_n\}$ of i.i.d samples the likelihood function is always a product:

$$p(D|\boldsymbol{\theta}) = \prod_{i=1}^{n} p(y_i|\theta)$$

- As we already saw it is very convinient to take the log of the likelihood, which gives:

$$\ln p(D|\boldsymbol{\theta}) = \ln \prod_{i=1}^{n} p(y_i|\theta) = \sum_{i=1}^{n} \ln p(y_i|\theta)$$

- very often we will be using the *negative log likelihood*, i.e.

$$-\ln p(D|\boldsymbol{\theta})$$

(similar to error, want to make it small)

Log Likelihood function of the Bernoulli

$$
\begin{aligned}
\ln \prod_{i=1}^{n} \mu_i^{y} (1-\mu)^{(1-y_i)} &= \sum_{i=1}^{n} \ln \mu^{y_i} (1-\mu)^{(1-y_i)} \\
&= \sum_{i=1}^{n} \ln \mu^{y_i} + \sum_{i=1}^{n} (1-\mu)^{(1-y_i)} \\
&= \sum_{i=1}^{n} y_i \ln \mu + \sum_{i=1}^{n} (1-y_i) \ln(1-\mu) \quad (14)
\end{aligned}
$$

Maximum Likelihood Estimation

- In statistics we do parameter estimation by *maximizing the likelihood*
- Maximum Likelihood Estimation (MLE) is general statistical estimation approach to estimate the parameters of a distribution/model given a dataset
- It looks for the parameters that maximize the dataset likelihood, i.e. in the case of the Bernoulli distribution:

$$\max_{\mu} p(D|\mu)$$

- We can choose to maximise the likelihood or the log likelihood, but we can also (equivalently) minimize the negative log likelihood.
- How to get the Maximum Likelihood Estimate of μ ?

Maximum Likelihood Estimation

- In statistics we do parameter estimation by *maximizing the likelihood*
- Maximum Likelihood Estimation (MLE) is general statistical estimation approach to estimate the parameters of a distribution/model given a dataset
- It looks for the parameters that maximize the dataset likelihood, i.e. in the case of the Bernoulli distribution:

$$\max_{\mu} p(D|\mu)$$

- We can choose to maximise the likelihood or the log likelihood, but we can also (equivalently) minimize the negative log likelihood.
- How to get the Maximum Likelihood Estimate of μ ? get the derivative of the likelihood, set it to zero and solve for the parameters μ

Maximum Log Likelihood Estimation for the Bernoulli

- find μ that maximises the log likelihood of the Bernoulli eq 14
- If you do it right you should get

$$\mu = \frac{1}{n} \sum_{i=1}^{n} y_i$$

For more on the Bernoulli distribution check [2] second chapter.

Back to Logistic regression

Logistic Regression, I

- We are given:
  - two classes C_1 and C_2 with respective class labels 1 and 0
  - $\mathbf{X}$ the matrix with lines the training instances, and
  - $\mathbf{y}$ the respective vector of class labels.

Logistic Regression, II

- The class y of some instance $\mathbf{x}$ follows a Bernoulli distribution.

  $$y \sim \text{Ber}(y|\mu(\mathbf{x})) = [p(y = C_1|\mu(\mathbf{x}))]^y [1 - p(y = C_1|\mu(\mathbf{x}))]^{1-y}$$

  now the "μ" is different for each instance $\mathbf{x}$
- Logistic regression will learn to estimate this conditional distribution, i.e. $P(y_i|\mu(\mathbf{x}_i))$, in particular it will learn to estimate $p(y = C_1|\mathbf{x})$ in the case of a two class problem.

Logistic Regression Likelihood Function

- The likelihood of the vector of observed class labels $\mathbf{y}$ is given by:

Logistic Regression Likelihood Function

- The likelihood of the vector of observed class labels $\mathbf{y}$ is given by:

$$
\begin{aligned}
p(\mathbf{y}|\mathbf{X}) &= \prod_{i=1}^{N} p(y_i|\mathbf{x}_i) \\
&= \prod_{i=1}^{N} [p(y_i = C_1|\mathbf{x}_i)]^{y_i} [1 - p(y_i = C_1|\mathbf{x}_i)]^{1-y_i}
\end{aligned}
$$

- Thus logistic regression will maximise the likelihood (in a similar manner to the simple case of μ) by learning to estimate $P(y|\mathbf{x})$

Logistic Regression, modelling $P(y|\mathbf{x})$, I

- As we said the label y of some instance $\boldsymbol{x}$:

$$y \sim \text{Ber}(y|\mu(\mathbf{x})), \text{ governed by } \mu(\boldsymbol{x}) := p(y = C_1|\mathbf{x})$$

- to simplify notation let:

$$p(C_i|\mathbf{x}) := p(y = C_i|\mathbf{x})$$

- where we compute $p(C_1|\mathbf{x})$ as:

$$p(C_1|\mathbf{x})$$

Logistic Regression, modelling $P(y|\mathbf{x})$, I

- As we said the label y of some instance $\boldsymbol{x}$:

$$y \sim \text{Ber}(y|\mu(\mathbf{x})), \text{ governed by } \mu(\boldsymbol{x}) := p(y = C_1|\mathbf{x})$$

- to simplify notation let:

$$p(C_i|\mathbf{x}) := p(y = C_i|\mathbf{x})$$

- where we compute $p(C_1|\mathbf{x})$ as:

$$\begin{aligned} p(C_1|\mathbf{x}) &= \frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_1)p(C_1) + p(\mathbf{x}|C_2)p(C_2)} \\ &= \frac{1}{1 + \exp(-a)} = \sigma(a) \end{aligned} \tag{15}$$

Logistic Regression, modelling $P(y|\mathbf{x})$, II I

- $\sigma(a)$ is the *logistic sigmoid function*, used also a lot in neural network algorithms.
- if we solve for a in eq 15 we get:

$$a = \ln \frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_2)p(C_2)} = \ln \frac{p(C_1|\mathbf{x})}{p(C_2|\mathbf{x})}$$

known as the *log odds ratio* or the *logit* function is the inverse of the $\sigma(a)$ function.

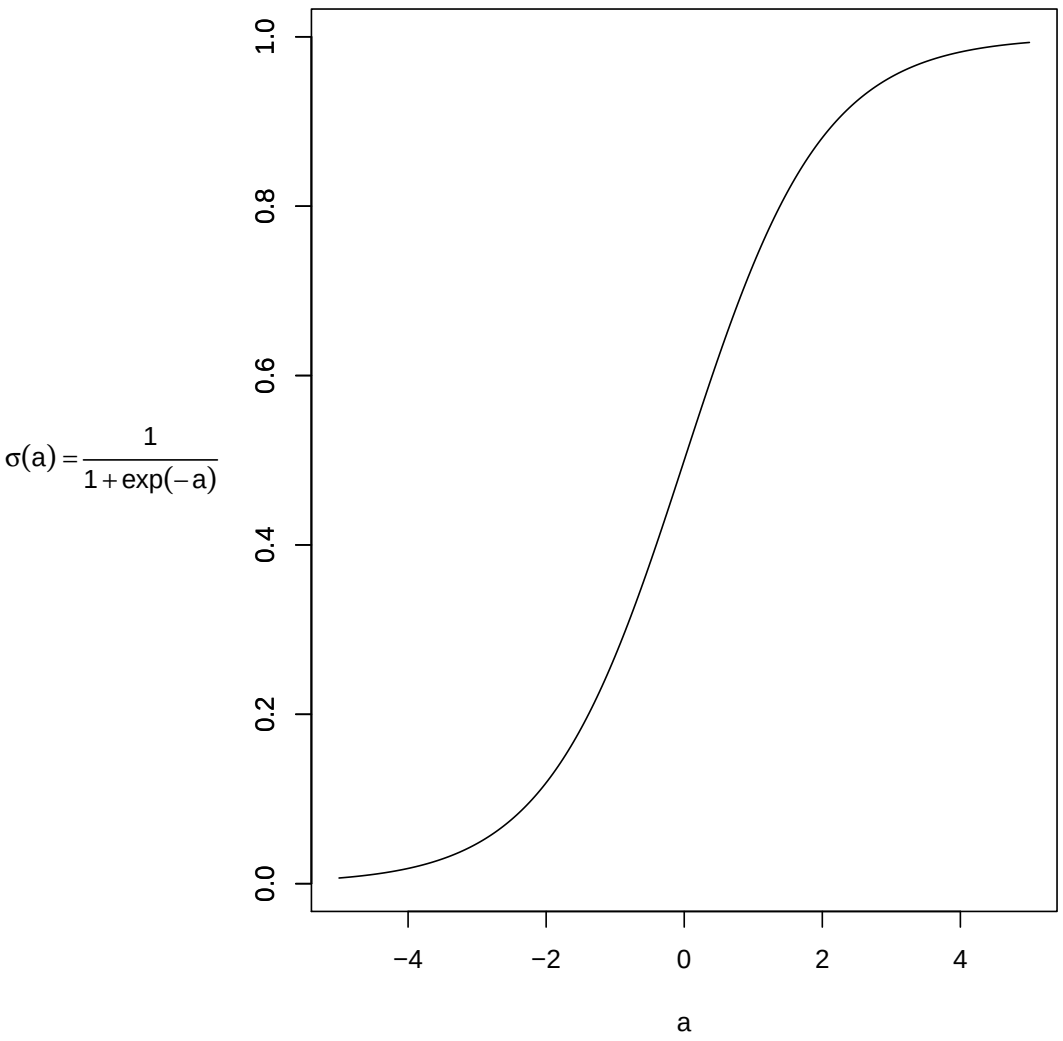
The logistic sigmoid function, $\sigma(a)$

- It squezes all the real axis in a finite interval.
- $\sigma(-a) = 1 - \sigma(a)$
- its inverse function is:

$$a = \ln \frac{\sigma(a)}{1 - \sigma(a)} = \ln \frac{p(C_1|\mathbf{x})}{p(C_2|\mathbf{x})}$$

the *logit function*, *log odds*

- $\frac{d\sigma(a)}{da} = \sigma(a)(1 - \sigma(a))$



A multiclass generalization of logistic sigmoid function

- When we have more than two classes, instead of the logistic sigmoid we use:

$$
\begin{aligned}
p(C_k|\mathbf{x}) &= \frac{p(\mathbf{x}|C_k)p(C_k)}{\sum_k p(\mathbf{x}|C_k)p(C_k)} \\
&= \frac{\exp a_k}{\sum_k \exp a_k}
\end{aligned}
$$

known as the *soft max*, $a_k = \ln p(\mathbf{x}|C_k)p(C_k)$

- What's the use of the re-writting we did?

$$
\begin{aligned}
p(C_1|\mathbf{x}) &= \frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_1)p(C_1) + p(\mathbf{x}|C_2)p(C_2)} \\
&= \frac{1}{1 + \exp(-a)} = \sigma(a)
\end{aligned}
$$

- What's the use of the re-writting we did?

$$\begin{aligned} p(C_1|\mathbf{x}) &= \frac{p(\mathbf{x}|C_1)p(C_1)}{p(\mathbf{x}|C_1)p(C_1) + p(\mathbf{x}|C_2)p(C_2)} \\ &= \frac{1}{1+\exp(-a)} = \sigma(a) \end{aligned}$$

we will make a a function of $\mathbf{x}$, and more precisely:

$$a(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$$

- for more than two classes and the softmax:

$$p(C_k|\mathbf{x}) = \frac{\exp a_k}{\sum_k \exp a_k}$$

we have:

$$a_k(\mathbf{x}) = \mathbf{w}_k^T\mathbf{x}$$

Logistic regression cost function

- Its cost function is, the negative log likelihood:

$$
\begin{aligned}
E(\mathbf{w}) &= -\ln p(\mathbf{y}|\mathbf{X}) = -\ln \prod_{i=1}^{N} p(C_1|\mathbf{x}_i)^{y_i}(1-p(C_1|\mathbf{x}_i))^{1-y_i} \\
&= -\sum_{i=1}^{N} y_i \ln p(C_1|\mathbf{x}_i) - \sum_{i=1}^{N}(1-y_i)\ln(1-p(C_1|\mathbf{x}_i)) \\
&= -\sum_{i=1}^{N} y_i \ln \sigma(\mathbf{w}^T\mathbf{x}_i) - \sum_{i=1}^{N}(1-y_i)\ln(1-\sigma(\mathbf{w}^T\mathbf{x}_i))
\end{aligned}
$$

which we minimize, thus we fall under the *Maximum Likelihood Estimation* paradigm. Its gradient is:

$$
\nabla E(\mathbf{w}) = \sum_{i=1}^{N}(\sigma(\mathbf{w}^T\mathbf{x}_i) - y_i)\mathbf{x}_i = \mathbf{X}^\mathsf{T}(\sigma(\mathbf{X}\mathbf{w}) - \mathbf{y})
$$

Logistic regression cost function/cross entropy

- if we denote by

$$Q_i(C|\sigma(\mathbf{x}_i)) = [P(C_1|\sigma(\mathbf{w}^T\mathbf{x}_i)), P(C_2|\sigma(\mathbf{w}^T\mathbf{x}_i))]$$

  the learned/predicted posterior of the class for the $\mathbf{x}_i$,
- and by

$$P_i(C|x_i) = [y_i, 1 - y_i]$$

  the true posterior,
- then we can rewritte the cost function as:

$$\sum_{i=1}^{N} H(P_i, Q_i)$$

- where

$$H(P, Q) = -\sum_{z} P(z) \ln Q(z)$$

  is the *cross entropy function* measuring distributions distance.
- Thus we seek $\mathbf{w}$ that minimizes the distance of the true from the learned posterior.

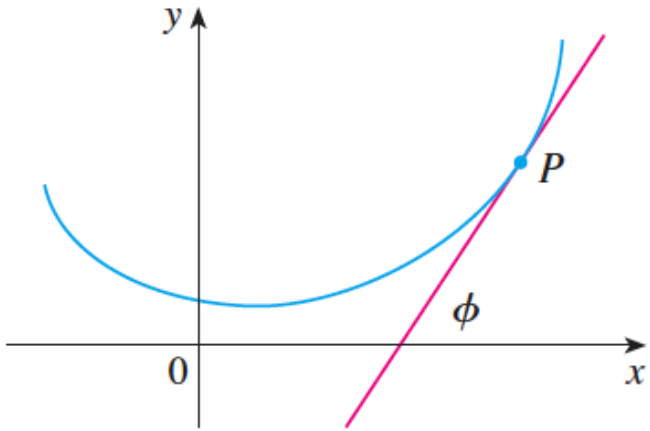
Logistic regression objective function and its relation to the objective functions of other methods

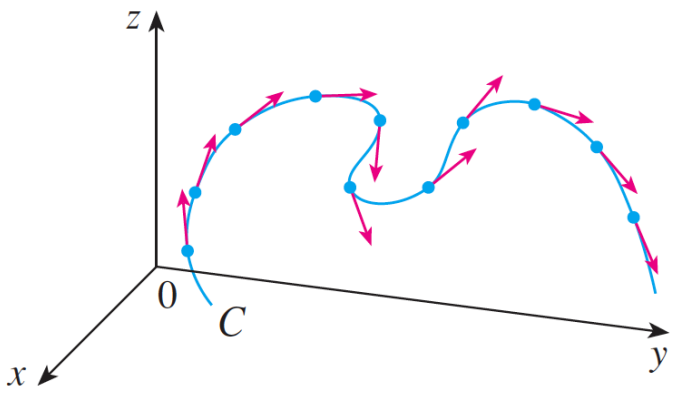
	Objective Function	Gradient
Perceptron	$\min_{\mathbf{w}} -\sum_{i=1}^{N} \mathbf{w}^{\mathsf{T}}\mathbf{x}_i(y_i - \text{sign}(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i)$	$\sum_{i=1}^{N} (\text{sign}(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i) - y_i)\mathbf{x}_i$
Log. R.	$\min_{\mathbf{w}} -\ln \prod_{i=1}^{N} p(C_1\|\mathbf{x}_i)^{y_i}(1 - p(C_2\|\mathbf{x}_i))^{1-y_i}$	$\sum_{i=1}^{N}(\sigma(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i) - y_i)\mathbf{x}_i$
Lin. R.	$\max_{\mathbf{w}} \ln \prod_{i=1}^{N} N(y_i\|\mathbf{w}^{T}\mathbf{x}_i, \sigma^2)$	$\sum_{i=1}^{N}(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i - y_i)\mathbf{x}_i$

- In perceptron the update is done for each missclassified instance
- In Log. R. the update is done according to the error of each instance
- Note the similarity of the gradient of Log.R. to that of Lin. R.
- In your implementations you should use the matrix variants of the gradients, see for example page 144 for logistic regression.

Optimizing the logistic regression cost function

- Obvious choice is the gradient descent
- Faster optimization methods take into account the *curvature* of the function





- The curvature at a point P is the rate of change of the angle ϕ with respect to the arc s, given by $|\frac{d\phi}{ds}|$, it is a measure of how fast the curve changes direction at P, [6].
- the second order derivative, Hessian, is a measure of the local curvature.

Newton's method

- Newton's method uses the curvature of the function through the Hessian
- such methods are called *second order* optimization methods

Reminder, Hessian

- For a function $f(\mathbf{x}) : R^d \to R$
- $\nabla^2 f(\mathbf{x})$ is the Hessian of f, which is the $d \times d$ matrix:

$$\nabla^2 f(\mathbf{x}) = \mathbf{H}(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_d} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_d} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_d \partial x_1} & \frac{\partial^2 f}{\partial x_d \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_d^2} \end{bmatrix}$$

Reminder, second order Taylor expansion

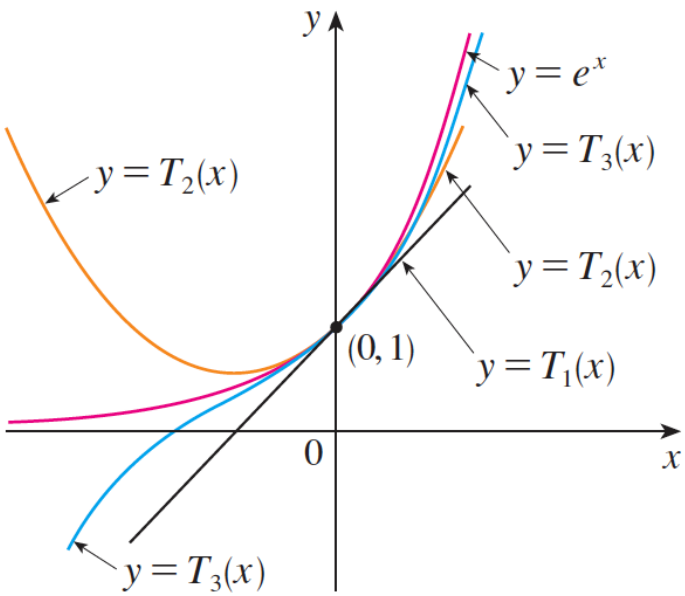
- The second order Taylor expansion of a function $f(.)$ on a point $\mathbf{z}$ around point $\mathbf{x}$, i.e. $\mathbf{z} = \mathbf{x} + \mathbf{v}$, $\mathbf{v} \to 0$, is given by

$$f(\mathbf{x} + \mathbf{v}) \simeq f(\mathbf{x}) + \nabla f(\mathbf{x})^T \mathbf{v} + \mathbf{v}^T \nabla^2 f(\mathbf{x}) \mathbf{v}$$

or

$$f(\mathbf{z}) \simeq f(\mathbf{x}) + \nabla f(\mathbf{x})^T (\mathbf{z} - \mathbf{x}) + (\mathbf{z} - \mathbf{x})^T \nabla^2 f(\mathbf{x}) (\mathbf{z} - \mathbf{x})$$

Examples of the first three order Taylor approximations of $f(x) = e^x$.



Newton's method

- Let $E(\mathbf{w})$ be a function which we want to optimize.
- Let $\mathbf{w}^{(k)}$ be the current estimate of the parameter vector.
- Then we can take its second order expansion around $\mathbf{w}^{(k)}$

$$\begin{aligned} E(\mathbf{w}) &= E(\mathbf{w}^{(k)} + \mathbf{h}) \\ &\simeq E(\mathbf{w}^{(k)}) + \nabla_{\mathbf{w}} E(\mathbf{w}^{(k)})^T \mathbf{h} + \frac{1}{2} \mathbf{h}^T \nabla^2_{\mathbf{w}} E(\mathbf{w}^{(k)}) \mathbf{h} \end{aligned}$$

- and search for that direction $\mathbf{h}$ that will optimize $E(\mathbf{w})$:

$$\max_{\mathbf{w}} / \min_{\mathbf{w}} E(\mathbf{w}) = \max_{\mathbf{h}} / \min_{\mathbf{h}} E(\mathbf{w}^{(k)} + \mathbf{h})$$

- In fact whether we $\min$ or $\max$ depends on the function $E(\mathbf{w})$

Newton's method

- Let $E(\mathbf{w})$ be a function which we want to optimize.
- Let $\mathbf{w}^{(k)}$ be the current estimate of the parameter vector.
- Then we can take its second order expansion around $\mathbf{w}^{(k)}$

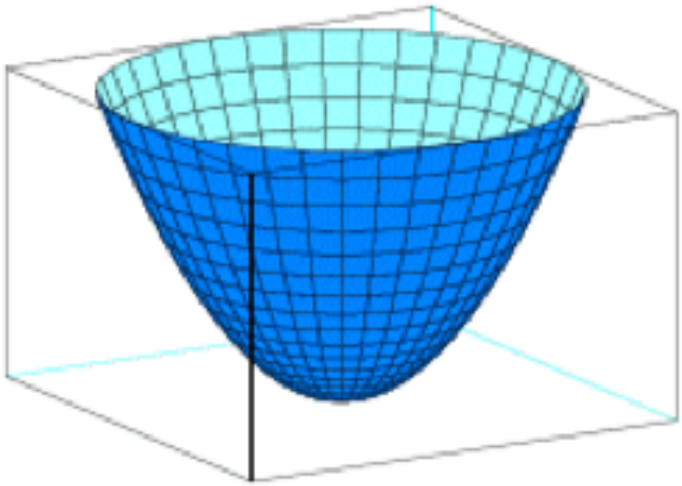
$$\begin{aligned} E(\mathbf{w}) &= E(\mathbf{w}^{(k)} + \mathbf{h}) \\ &\simeq E(\mathbf{w}^{(k)}) + \nabla_{\mathbf{w}} E(\mathbf{w}^{(k)})^T \mathbf{h} + \frac{1}{2}\mathbf{h}^T \nabla^2_{\mathbf{w}} E(\mathbf{w}^{(k)})\mathbf{h} \end{aligned}$$

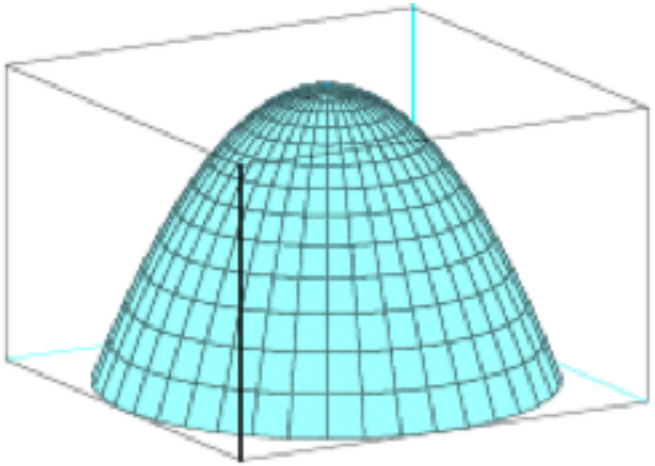
- and search for that direction $\mathbf{h}$ that will optimize $E(\mathbf{w})$:

$$\max_{\mathbf{w}} / \min_{\mathbf{w}} E(\mathbf{w}) = \max_{\mathbf{h}} / \min_{\mathbf{h}} E(\mathbf{w}^{(k)} + \mathbf{h})$$

- In fact whether we $\min$ or $\max$ depends on the function $E(\mathbf{w})$
  - If $\mathbf{x}^T \nabla^2 E(\mathbf{w})\mathbf{x} \geq 0, \forall x$ then $E(\mathbf{w})$ is convex and has a unique minimum
  - If $-\mathbf{x}^T \nabla^2 E(\mathbf{w})\mathbf{x} \geq 0, \forall x$ then $E(\mathbf{w})$ is concave and has a unique maximum

Newton's method (Convex vs Concave)





convex $f(\mathbf{w}) = \mathbf{w}^T\mathbf{w}$

$$\nabla f(\mathbf{w}) = \mathbf{w}$$

$$\nabla^2 f(\mathbf{w}) = \mathbf{I}$$

concave $f(\mathbf{w}) = -\mathbf{w}^T\mathbf{w}$

$$\nabla f(\mathbf{w}) = -\mathbf{w}$$

$$\nabla^2 f(\mathbf{w}) = -\mathbf{I}$$

Newton's method

- The optimizing direction $\mathbf{h}$ is given by:

Newton's method

- The optimizing direction $\mathbf{h}$ is given by:

$$\begin{aligned}\nabla_{\mathbf{h}} E(\mathbf{w}^{(k)} + \mathbf{h}) = \mathbf{0} \quad &\Rightarrow \quad \nabla_{\mathbf{w}} E(\mathbf{w}^{(k)}) + \nabla^2_{\mathbf{w}^{(k)}} E(\mathbf{w}^{(k)})\mathbf{h} = \mathbf{0} \\ &\Rightarrow \quad \mathbf{h} = -(\nabla^2_{\mathbf{w}} E(\mathbf{w}^{(k)}))^{-1} \nabla_{\mathbf{w}} E(\mathbf{w}^{(k)}) \\ &\qquad\quad = -(\mathbf{H}(\mathbf{w}^{(k)}))^{-1} \nabla_{\mathbf{w}} E(\mathbf{w}^{(k)})\end{aligned}$$

- thus:

$$\mathbf{w}^{(k+1)} := \mathbf{w}^{(k)} + \eta \mathbf{h}$$

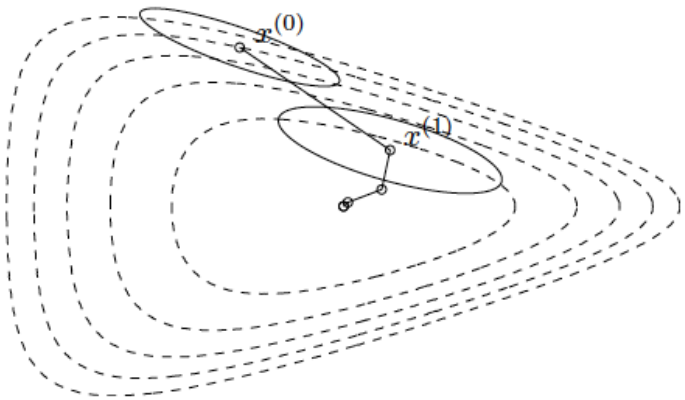
  where η is the learning rate,
- note that it is the sign of $\mathbf{H}(\mathbf{w}^{(k)})$ that indicates whether we will move in the direction of the gradient or in the opposite.
  - if $\mathbf{x}^T \mathbf{H}(\mathbf{w}^{(k)}) \mathbf{x} \geq 0$ (i.e. $\mathbf{H}$ PSD) then $\mathbf{h}$ moves to the oposite of the gradient, to a minimum point.
  - if $-\mathbf{x}^T \mathbf{H}(\mathbf{w}^{(k)}) \mathbf{x} \geq 0$ (i.e. $\mathbf{H}$ NSD) then $\mathbf{h}$ moves to the direction of the gradient, to a maximum point.

Newton's method and the Hessian, [3]

- In Newton's method the optimization direction is $-(\mathbf{H}(\mathbf{w}^{(k)}))^{-1}\nabla_{\mathbf{w}}E(\mathbf{w}^{(k)})$, i.e. we project the gradient on the inverse of the Hessian.
- This is equivalent to a change of coordinates given by $\mathbf{H}(\mathbf{w}^{(k)}))^{1/2}\mathbf{w}$, and thus the definition of a new norm given by: $\mathbf{w}^T(H(\mathbf{w}^{(k)})\mathbf{w}$. Newton does stepest gradient ascent/descent on the new norm.
- Thus we make large steps in the directions that the gradient does not change much (small second derivative) and small steps when the gradient changes a lot (large second derivative).

- Solid lines in the fig. mark the unit ellipsoids centered at $x^{(k)}$ under the Hessian norm, i.e.

$$(x-x^{(k)})^T H(x^{(k)})(x-x^{(k)})$$



Newton's method vs gradient descent, [4]

- Newton gives faster improvement even compared with having the optimal η
- Newton is not applicable when the Hessian is singular
- However doing a matrix inversion $O(d^3)$ at each step can offset the descent advantage.

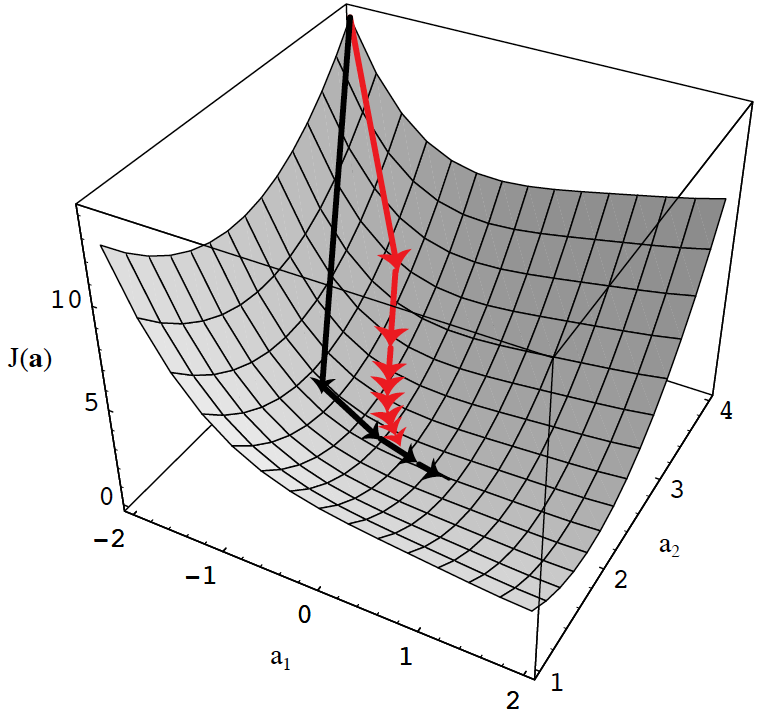
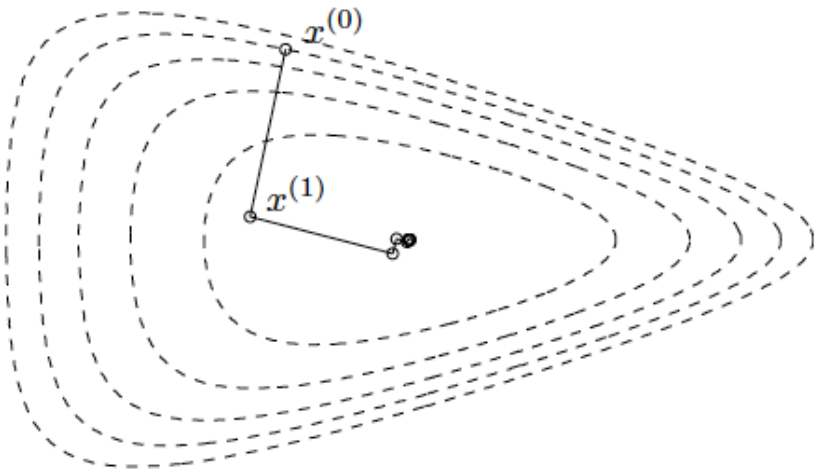
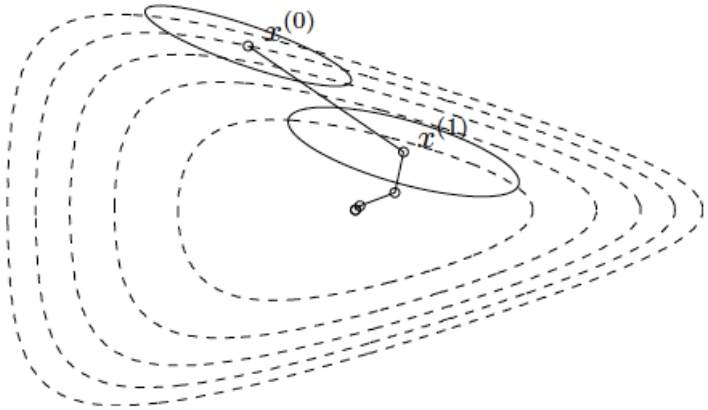


Figure: Sequence of weight vectors given by: gradient descent, black Newton

Newton's method vs gradient descent, [3]





Contour plots of a given function. Left gradient descent steps with exact line search, right Newton's method and backtracking line search.

Newton's method for logistic regression

- Error function:

$$E(\mathbf{w}) = -\sum_{i=1}^{N} y_i \ln \sigma(\mathbf{w}^T\mathbf{x}_i) - \sum_{i=1}^{N}(1-y_i)\ln(1-\sigma(\mathbf{w}^T\mathbf{x}_i))$$

- Gradient:

$$\nabla E(\mathbf{w}) = \sum_{i=1}^{N}(\sigma(\mathbf{w}^T\mathbf{x}_i) - y_i)\mathbf{x}_i = \mathbf{X}^T(\boldsymbol{\sigma} - \mathbf{y})$$

where $\boldsymbol{\sigma} := (\sigma(\mathbf{w}^T\mathbf{x}_1), \ldots, \sigma(\mathbf{w}^T\mathbf{x}_n))^T$

- Hessian:

$$\mathbf{H}(\mathbf{w}) = \nabla\nabla E(\mathbf{w}) = \sum_{i=1}^{N} \sigma_i(1-\sigma_i)\mathbf{x}_i\mathbf{x}_i^T = \mathbf{X}^T\mathbf{R}\mathbf{X}$$

where $\mathbf{R} = diag(\sigma_1(1-\sigma_1), \ldots, \sigma_n(1-\sigma_n))$. Any idea what the quantity $\sigma_i(1-\sigma_i)$?

Newton's method for logistic regression

- Error function:

$$E(\mathbf{w}) = -\sum_{i=1}^{N} y_i \ln \sigma(\mathbf{w}^T\mathbf{x}_i) - \sum_{i=1}^{N}(1-y_i)\ln(1-\sigma(\mathbf{w}^T\mathbf{x}_i))$$

- Gradient:

$$\nabla E(\mathbf{w}) = \sum_{i=1}^{N}(\sigma(\mathbf{w}^T\mathbf{x}_i) - y_i)\mathbf{x}_i = \mathbf{X}^T(\boldsymbol{\sigma} - \mathbf{y})$$

where $\boldsymbol{\sigma} := (\sigma(\mathbf{w}^T\mathbf{x}_1), \ldots, \sigma(\mathbf{w}^T\mathbf{x}_n))^T$

- Hessian:

$$\mathbf{H}(\mathbf{w}) = \nabla\nabla E(\mathbf{w}) = \sum_{i=1}^{N} \sigma_i(1-\sigma_i)\mathbf{x}_i\mathbf{x}_i^T = \mathbf{X}^T\mathbf{R}\mathbf{X}$$

where $\mathbf{R} = diag(\sigma_1(1-\sigma_1), \ldots, \sigma_n(1-\sigma_n))$. Any idea what the quantity $\sigma_i(1-\sigma_i)$? The variance of a Bernouli distribution.

Newton's method for logistic regression

- The update rule is:

$$\begin{aligned}
\mathbf{w}^{(k+1)} &= \mathbf{w}^{(k)} - \mathbf{H}(\mathbf{w}^{(k)})^{-1}\nabla E(\mathbf{w}^{(k)}) \\
&= \mathbf{w}^{(k)} - (\mathbf{X}^T\mathbf{R}\mathbf{X})^{-1}\mathbf{X}^T(\boldsymbol{\sigma} - \mathbf{y}) \\
&= (\mathbf{X}^T\mathbf{R}\mathbf{X})^{-1}[(\mathbf{X}^T\mathbf{R}\mathbf{X})\mathbf{w}^{(k)} - \mathbf{X}^T(\boldsymbol{\sigma} - \mathbf{y})] \\
&= (\mathbf{X}^T\mathbf{R}\mathbf{X})^{-1}\mathbf{X}^T\mathbf{R}[\mathbf{X}\mathbf{w}^{(k)} - \mathbf{R}^{-1}(\boldsymbol{\sigma} - \mathbf{y})]
\end{aligned}$$

- setting $\mathbf{z} := \mathbf{X}\mathbf{w}^{(k)} - \mathbf{R}^{-1}(\boldsymbol{\sigma} - \mathbf{y})$ we can rewrite as:

$$\mathbf{w}^{(k+1)} = (\mathbf{X}^T\mathbf{R}\mathbf{X})^{-1}\mathbf{X}^T\mathbf{R}\mathbf{z}$$

which is known as *iterative reweighted least squares* where at each step k we recompute the $\mathbf{R}$ weighting matrix.

Least squares/reweighted least squares

- Standard least squares, optimization problem

$$\min_{\mathbf{w}} \sum_{i}^{N} (y_i - \mathbf{w}^T \mathbf{x}_i)^2 = \min_{\mathbf{w}} (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

- Standard least squares, solution:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- reweighted least squares, optimization problem

$$\min_{\mathbf{w}} \sum_{i}^{N} R_i (y_i - \mathbf{w}^T \mathbf{x}_i)^2 = \min_{\mathbf{w}} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{R} (\mathbf{y} - \mathbf{X}\mathbf{w})$$

- reweighted least squares, solution:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{R} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{R} \mathbf{y}$$

the contribution of the ith instance is weighted by R_i which as we saw was the variance of its prediction.

On the Hessian/cost function of the logistic regression

$$\begin{aligned}\mathbf{H}(\mathbf{w}) &= \nabla\nabla E(\mathbf{w}) = \sum_{i=1}^{N} \sigma_i(1-\sigma_i)\mathbf{x}_i\mathbf{x}_i^T \\ &= \mathbf{X}^T\mathbf{R}\mathbf{X}\end{aligned}$$

- The Hessian is a function of $\mathbf{w}$ and *not a constant*
- $\forall\ \mathbf{z}, \mathbf{z}^T\mathbf{H}\mathbf{z} > 0$ thus $\mathbf{H}$ is positive definite (except when $||\mathbf{w}|| \to \infty$, in which case $\mathbf{H} \to 0$, comments bellow).
- and $E(\mathbf{w})$ is a convex function with a unique minimum.

Logistic regression cost function drawbacks [5, 2] I

- It uses the Maximum Likelihood principle
- as all ML methods it is prone to overfitting
- In particular if the problem is linearly seperable
  - then the negative likelihood is minimized by finding the direction $\mathbf{w}$ for which $\sigma(\mathbf{w}^\mathsf{T}\mathbf{x}) = 1/2$ i.e. $\mathbf{w}^\mathsf{T}\mathbf{x} = 0$ and sending $||\mathbf{w}|| \to \infty$
  - then $P(C_1|\mathbf{x}) = 1, \forall \mathbf{x} \in C_1)$ and $P(C_2|\mathbf{x}) = 0, \forall \mathbf{x} \in C_2)$
  - and the sigmoid function converges to a Heavyside function in the feature space.
  - we also have: $\sigma_i(1 - \sigma_i) \to 0$ thus $\mathbf{H} \to 0$.
- this assigns maximum probability mass in the training samples but does not generalise,
- and there is a continium of such equivalent solutions which we cannot distinguish

Logistic regression cost function drawbacks [5, 2] II

- Solution:
  - Include a prior on the model parameters, e.g. Gaussian/Laplacian, thus going the Maximum A Posteriori way
  - or equivalently use a regularization terms such as $||\mathbf{w}||_2^2$ or $||\mathbf{w}||_1$

Bibliography I

Mokhtar S Bazaraa, Hanif D Sherali, and C M Shetty.
Nonlinear Programming: Theory and Algorithms; 3rd ed.
Wiley, Newark, NJ, 2006.

Christopher M. Bishop.
Pattern recognition and machine learning, 5th Edition.
Information science and statistics. Springer, 2007.

Stephen Boyd and Lieven Vandenberghe.
Convex Optimization.
Cambridge University Press, New York, NY, USA, 2004.

Richard O. Duda, Peter E. Hart, and David G. Stork.
Pattern Classification.
Wiley-Interscience, 2001.

Bibliography II

Kevin Murphy.
Machine Learning: A probabilistic Perspective.
MIT, 2012.

James Stewart.
Calculus, second edition.
Thomson Information Publishing Group, 1991.